

M66 Scenario Simulation

This notebook evaluates the M66 passenger queue simulation under the reference condition and three stress-test scenarios. All scenarios use the same discrete-event simulation logic so that differences in the results come from the defined demand and service changes rather than from different simulation engines.

Scenarios

Scenario	Passenger Demand	Service Condition
Baseline	Baseline selected-stop demand	Published GTFS reference service
High-Demand	+40% boarding demand at the five modeled stops	Baseline GTFS service
Reduced-Service	Baseline selected-stop demand	One reference trip removed from each study hour
Combined	+40% boarding demand at the five modeled stops	Reduced-Service schedule

For High-Demand, the +40% increase applies only to boarding demand at the five modeled stops. Baseline GTFS service, upstream initial occupancy, and alighting targets remain unchanged.

For Reduced-Service, one reference trip is removed from every study hour using the documented deterministic removal rule. Initial occupancy and alighting targets are recomputed for the remaining trips.

For Combined, the High-Demand passenger streams are used together with the Reduced-Service schedule. The Reduced-Service occupancy and alighting inputs are reused and are not multiplied by 1.40 because the demand increase is limited to boarding demand at the five modeled stops.

All scenarios start with 30 matched replications. Replications may be extended in batches of 10, up to 50, if the required 95% confidence-interval precision is not met.

Notebook Role

Purpose: Evaluate the Baseline, High-Demand, Reduced-Service, and Combined reference conditions using the shared queueing model and matched stochastic replications.

Run order: 3 of 4

Main outputs: scenario inputs, replication results, precision checks, stop-level results, validation tables, and scenario figures.

Technical reference: `../docs/TECHNICAL_DOCUMENTATION.md`

```
In [1]: from pathlib import Path
import heapq

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.stats import t
```

1. Setup and Configuration

```
In [2]: # Locate the MODESIM_Project directory from common notebook launch locations.
working_dir = Path.cwd()

project_candidates = [
    working_dir,
    working_dir.parent,
    working_dir / "MODESIM_Project",
]

project_dir = next(
    (
        path
        for path in project_candidates
        if (path / "code").exists() and (path / "data").exists()
    ),
    None,
)
```

```

if project_dir is None:
    raise FileNotFoundError(
        "Could not locate MODESIM_Project. "
        "Run the notebook from the repository root or MODESIM_Project/code."
    )

raw_dir = project_dir / "data" / "raw"
processed_dir = project_dir / "data" / "processed"
gtfs_dir = raw_dir / "gtfs_m"

figures_dir = project_dir / "output" / "figures"
tables_dir = project_dir / "output" / "tables"

clean_file = processed_dir / "MTA_M66_Selected_Stops_2025_CLEAN.csv"
raw_file = raw_dir / "MTA_M66_Eastbound_2025_RAW.csv"

figures_dir.mkdir(parents=True, exist_ok=True)
tables_dir.mkdir(parents=True, exist_ok=True)

required_inputs = [
    clean_file,
    raw_file,
    gtfs_dir / "trips.txt",
    gtfs_dir / "stop_times.txt",
]

missing_inputs = [path for path in required_inputs if not path.exists()]

if missing_inputs:
    missing_list = "\n".join(f"- {path}" for path in missing_inputs)
    raise FileNotFoundError(
        f"Required simulation inputs are missing:\n{missing_list}"
    )

print("Project directory located successfully.")
print("Required simulation inputs found.")

```

Project directory located successfully.

Required simulation inputs found.

```
In [3]: # Core simulation configuration
bus_capacity = 60

morning_hours = [6, 7, 8, 9, 10]
evening_hours = [15, 16, 17, 18, 19]
study_hours = morning_hours + evening_hours

selected_stops = [6, 7, 8, 9, 10]

master_seed = 2026

# Scenario parameters
demand_growth_factor = 1.40
trips_removed_per_hour = 1

# Replication and precision parameters
precision_start_replications = 30
precision_increment = 10
precision_max_replications = 50
precision_confidence_level = 0.95
```

2. Load Passenger Demand

```
In [4]: # Load the cleaned M66 selected-stop dataset used as the passenger-demand input
# for all simulation scenarios.
df = pd.read_csv(clean_file)

# Convert the date and hour fields into consistent datetime/hour values so the
# records can be checked against the defined study periods.
df["Date"] = pd.to_datetime(df["Date"])
df["Hour"] = pd.to_datetime(df["Hour"]).dt.hour

# Collect the study hours and stop sequences that are actually present in the
# processed dataset.
actual_hours = sorted(df["Hour"].unique().tolist())
actual_stops = sorted(df["Stop Sequence"].unique().tolist())

# Fail early if the loaded dataset does not contain exactly the study hours
```

```

# expected by the simulation configuration.
if actual_hours != study_hours:
    raise ValueError(
        f"Unexpected study hours. Expected {study_hours}, found {actual_hours}."
    )

# Fail early if the loaded dataset contains a different set of modeled stops.
# This prevents the simulation from silently running with the wrong input data.
if actual_stops != selected_stops:
    raise ValueError(
        f"Unexpected selected stops. Expected {selected_stops}, "
        f"found {actual_stops}."
    )

# Display a compact input summary so a reviewer can confirm that the intended
# processed dataset was loaded before the simulation continues.
print(f"Cleaned rows: {len(df):,}")
print(f"Study hours: {actual_hours}")
print(f"Selected stops: {actual_stops}")

```

Cleaned rows: 12,814

Study hours: [6, 7, 8, 9, 10, 15, 16, 17, 18, 19]

Selected stops: [6, 7, 8, 9, 10]

```

In [5]: # Reference: pandas DataFrame.groupby()
# https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.groupby.html
#
# Representative mean Boardings/Alightings by stop-hour, and the baseline
# whole-number Passenger Count used by the simulation. Identical rounding
# rule as the baseline notebook: max(0, floor(p + 0.5)).

demand_table = (
    df.groupby(["Stop Sequence", "Stop Name", "Hour"])[["Boardings", "Alightings"]]
    .mean()
    .reset_index()
    .sort_values(["Stop Sequence", "Hour"])
    .reset_index(drop=True)
)

baseline_demand = demand_table[
    ["Stop Sequence", "Stop Name", "Hour", "Boardings"]

```

```

].copy()

baseline_demand["Passenger Count"] = np.maximum(
    0,
    np.floor(baseline_demand["Boardings"] + 0.5)
).astype(int)

print("Baseline passenger total per replication:", baseline_demand["Passenger Count"].sum())
baseline_demand.head(10)

```

Baseline passenger total per replication: 776

Out[5]:

	Stop Sequence	Stop Name	Hour	Boardings	Passenger Count
0	6 W 65 ST/CENTRAL PARK WEST		6	7.543307	8
1	6 W 65 ST/CENTRAL PARK WEST		7	25.386100	25
2	6 W 65 ST/CENTRAL PARK WEST		8	40.393822	40
3	6 W 65 ST/CENTRAL PARK WEST		9	24.162791	24
4	6 W 65 ST/CENTRAL PARK WEST		10	18.968750	19
5	6 W 65 ST/CENTRAL PARK WEST		15	33.683398	34
6	6 W 65 ST/CENTRAL PARK WEST		16	28.030888	28
7	6 W 65 ST/CENTRAL PARK WEST		17	31.360153	31
8	6 W 65 ST/CENTRAL PARK WEST		18	22.000000	22
9	6 W 65 ST/CENTRAL PARK WEST		19	11.316206	11

3. Build High-Demand Passenger Demand

The High-Demand scenario increases boarding demand by 40% at each modeled stop-hour while keeping the same stops, study hours, and Baseline service conditions.

For a Baseline stop-hour boarding value (p), the High-Demand passenger count is calculated as:

$$[p_{HD}] = \max(0, \lfloor 1.40p + 0.5 \rfloor)$$

The `+0.5` before applying the floor operation implements the same half-up whole-passenger rounding rule used for the Baseline demand inputs. Because rounding is performed separately for every stop-hour cell, the realized total increase may differ slightly from exactly 40%.

For matched scenario comparisons, the original Baseline passengers are preserved. Only the additional passengers required to reach the High-Demand count are generated separately using a deterministic random sub-stream. This allows the Baseline portion of the passenger stream to remain unchanged between Baseline and High-Demand replications.

```
In [6]: # Build the High-Demand stop-hour table from the same demand inputs used
# for the Baseline scenario.
high_demand_demand = demand_table[
    ["Stop Sequence", "Stop Name", "Hour", "Boardings"]
].copy()

# Increase boarding demand by the configured High-Demand multiplier.
high_demand_demand["Scaled Boardings"] = (
    high_demand_demand["Boardings"] * demand_growth_factor
)

# Convert scaled boarding demand to whole passengers using the same
# half-up rounding rule used for the Baseline passenger counts.
high_demand_demand["Passenger Count"] = np.maximum(
    0,
    np.floor(high_demand_demand["Scaled Boardings"] + 0.5)
).astype(int)

# Compare the Baseline and High-Demand totals and report the realized
# demand increase after stop-hour rounding.
baseline_total = baseline_demand["Passenger Count"].sum()
high_demand_total = high_demand_demand["Passenger Count"].sum()

print("Baseline passenger total per replication:", baseline_total)
print("High-Demand passenger total per replication:", high_demand_total)
print(
    "Realized demand growth:",
    round(high_demand_total / baseline_total, 4)
)
```

```
high_demand_demand.head(10)
```

Baseline passenger total per replication: 776

High-Demand passenger total per replication: 1083

Realized demand growth: 1.3956

Out[6]:

	Stop Sequence	Stop Name	Hour	Boardings	Scaled Boardings	Passenger Count
0	6 W 65 ST/CENTRAL PARK WEST		6	7.543307	10.560630	11
1	6 W 65 ST/CENTRAL PARK WEST		7	25.386100	35.540541	36
2	6 W 65 ST/CENTRAL PARK WEST		8	40.393822	56.551351	57
3	6 W 65 ST/CENTRAL PARK WEST		9	24.162791	33.827907	34
4	6 W 65 ST/CENTRAL PARK WEST		10	18.968750	26.556250	27
5	6 W 65 ST/CENTRAL PARK WEST		15	33.683398	47.156757	47
6	6 W 65 ST/CENTRAL PARK WEST		16	28.030888	39.243243	39
7	6 W 65 ST/CENTRAL PARK WEST		17	31.360153	43.904215	44
8	6 W 65 ST/CENTRAL PARK WEST		18	22.000000	30.800000	31
9	6 W 65 ST/CENTRAL PARK WEST		19	11.316206	15.842688	16

```
In [7]: # Calculate the additional passengers required in each stop-hour cell.
# The Baseline passenger counts are kept separately for matched scenario runs.
additional_demand = baseline_demand[
    ["Stop Sequence", "Stop Name", "Hour", "Passenger Count"]
].rename(
    columns={"Passenger Count": "Baseline Passenger Count"}
).merge(
    high_demand_demand[
        ["Stop Sequence", "Stop Name", "Hour", "Passenger Count"]
    ].rename(
        columns={"Passenger Count": "High-Demand Passenger Count"}
    ),
    on=["Stop Sequence", "Stop Name", "Hour"]
)
```

```

additional_demand["Additional Passenger Count"] = (
    additional_demand["High-Demand Passenger Count"]
    - additional_demand["Baseline Passenger Count"]
)

# Verify that applying the High-Demand multiplier never decreases the
# passenger count of any Baseline stop-hour cell.
assert (additional_demand["Additional Passenger Count"] >= 0).all(), (
    "Every High-Demand stop-hour count must be greater than or equal "
    "to its Baseline count."
)

# Report the number of new passengers that must be generated in addition
# to the unchanged Baseline passenger stream.
additional_passenger_total = additional_demand[
    "Additional Passenger Count"
].sum()

print(
    "Additional passengers to generate per replication:",
    additional_passenger_total
)

additional_demand.head(10)

```

Additional passengers to generate per replication: 307

Out[7]:

	Stop Sequence	Stop Name	Hour	Baseline Passenger Count	High-Demand Passenger Count	Additional Passenger Count
0	6	W 65 ST/CENTRAL PARK WEST	6	8	11	3
1	6	W 65 ST/CENTRAL PARK WEST	7	25	36	11
2	6	W 65 ST/CENTRAL PARK WEST	8	40	57	17
3	6	W 65 ST/CENTRAL PARK WEST	9	24	34	10
4	6	W 65 ST/CENTRAL PARK WEST	10	19	27	8
5	6	W 65 ST/CENTRAL PARK WEST	15	34	47	13
6	6	W 65 ST/CENTRAL PARK WEST	16	28	39	11
7	6	W 65 ST/CENTRAL PARK WEST	17	31	44	13
8	6	W 65 ST/CENTRAL PARK WEST	18	22	31	9
9	6	W 65 ST/CENTRAL PARK WEST	19	11	16	5

4. Load GTFS Reference Service

The reference service is taken from the frozen MTA static GTFS data included with the project.

The simulation uses weekday eastbound M66 trips on the full-route shape `M660030`. Short-turn trips are excluded so that every modeled trip serves all five selected stops, Stop Sequences 6 through 10.

A trip is assigned to a study hour using its scheduled arrival time at Stop Sequence 6, the first modeled stop. The published GTFS stop times are retained for the reference scenarios.

```
In [8]: # Load the GTFS trip and stop-time tables used to construct the
# reference M66 service schedule.
trips = pd.read_csv(gtfs_dir / "trips.txt")
stop_times = pd.read_csv(gtfs_dir / "stop_times.txt")

# Keep weekday eastbound M66 trips that use the selected full-route shape.
# This excludes short-turn service that does not follow the full study segment.
m66_trips = trips[
    (trips["route_id"] == "M66")
    & (trips["service_id"] == "MQ_C6-Weekday")
    & (trips["direction_id"] == 0)
    & (trips["shape_id"] == "M660030")
].copy()

if m66_trips.empty:
    raise ValueError(
        "No weekday eastbound M66 full-route trips were found in the GTFS input."
    )

print("Full-route weekday trips:", len(m66_trips))
```

Full-route weekday trips: 138

```
In [9]: # Keep GTFS stop-time records for the five modeled stops.
selected_stop_times = stop_times[
    stop_times["trip_id"].isin(m66_trips["trip_id"])
    & stop_times["stop_sequence"].between(6, 10)
].copy()

# Use Stop Sequence 6 as the reference point for assigning each trip
# to a study hour.
first_stop_times = selected_stop_times[
    selected_stop_times["stop_sequence"] == 6
].copy()

first_stop_times["Hour"] = (
    first_stop_times["arrival_time"]
```

```

        .str.split(":")
        .str[0]
        .astype(int)
    )

    # Identify trips whose Stop 6 arrival occurs within the defined
    # Morning or Evening study hours.
    study_trip_ids = first_stop_times[
        first_stop_times["Hour"].isin(study_hours)
    ]["trip_id"]

    study_stop_times = selected_stop_times[
        selected_stop_times["trip_id"].isin(study_trip_ids)
    ].copy()

    # Convert GTFS arrival times to minutes after midnight for use by the
    # simulation and the Reduced-Service trip-removal rule.
    def time_to_minutes(time_text):
        """Convert a GTFS HH:MM:SS time value to minutes after midnight."""
        hours, minutes, seconds = map(int, time_text.split(":"))
        return (hours * 60) + minutes + (seconds / 60)

    study_stop_times["arrival_minutes"] = (
        study_stop_times["arrival_time"].apply(time_to_minutes)
    )

    first_stop_times["Arrival Minutes"] = (
        first_stop_times["arrival_time"].apply(time_to_minutes)
    )

    # Count reference trips according to their scheduled Stop 6 arrival hour.
    trips_per_hour = (
        first_stop_times[
            first_stop_times["Hour"].isin(study_hours)
        ]
        .groupby("Hour")
        .size()
        .reindex(study_hours, fill_value=0)
    )

```

```

# Verify that each selected trip has one stop-time record at all five
# modeled stops.
expected_stop_events = study_trip_ids.nunique() * len(selected_stops)

if len(study_stop_times) != expected_stop_events:
    raise ValueError(
        "The GTFS study service does not contain exactly one record "
        "for every modeled stop on every selected trip."
    )

print("Study-period trips:", study_trip_ids.nunique())
print("Study-period stop events:", len(study_stop_times))
print("\nBaseline trips per hour:")
print(trips_per_hour)

```

Study-period trips: 87

Study-period stop events: 435

Baseline trips per hour:

Hour

6	5
7	12
8	12
9	8
10	6
15	8
16	12
17	11
18	8
19	5

dtype: int64

In [10]: # Build the hourly Baseline service plan from the reference GTFS trip counts.

```

baseline_service_plan = trips_per_hour.reset_index(name="Trips")

baseline_service_plan["Period"] = np.where(
    baseline_service_plan["Hour"] < 12,
    "Morning",
    "Evening"
)

```

```
# Calculate the fixed Morning and Evening trip budgets used for
# reference-service comparisons.
service_budget = (
    baseline_service_plan
    .groupby("Period")["Trips"]
    .sum()
    .to_dict()
)

print(
    "Baseline Morning service budget:",
    service_budget["Morning"]
)
print(
    "Baseline Evening service budget:",
    service_budget["Evening"]
)

baseline_service_plan
```

Baseline Morning service budget: 43

Baseline Evening service budget: 44

Out[10]:

	Hour	Trips	Period
0	6	5	Morning
1	7	12	Morning
2	8	12	Morning
3	9	8	Morning
4	10	6	Morning
5	15	8	Evening
6	16	12	Evening
7	17	11	Evening
8	18	8	Evening
9	19	5	Evening

5. Build Reduced-Service Reference Schedule

The Reduced-Service scenario removes exactly one published GTFS trip from every study hour. Passenger demand remains at the Baseline level.

Trip removal follows a deterministic closest-to-midpoint rule:

1. For each study hour, scheduled trips are identified using their Stop Sequence 6 arrival times.
2. The midpoint of the hour is defined as `HH:30`.
3. The trip with the smallest absolute time difference from the midpoint is removed.
4. If two trips are equally close, the earlier scheduled trip is removed.
5. The selected `trip_id` is removed from all five modeled stops so that no partial trip remains.

The remaining trips retain their original published GTFS arrival times. No synthetic service times are created for the Reduced-Service reference schedule.

Removing one trip from each of the five Morning hours reduces the Morning service budget from 43 to 38 trips. Removing one trip from each of the five Evening hours reduces the Evening service budget from 44 to 39 trips.

```
In [11]: # Select one trip for removal from each study hour using the
# deterministic closest-to-midpoint rule.
removed_records = []

for hour in study_hours:
    hour_trips = first_stop_times[
        first_stop_times["Hour"] == hour
    ].copy()

    if hour_trips.empty:
        raise ValueError(
            f"No reference trip is available for study hour {hour}."
        )

    hour_midpoint_minutes = (hour * 60) + 30

    # Calculate each trip's absolute distance from the midpoint
    # of its assigned study hour.
    hour_trips["Distance to Midpoint"] = (
        hour_trips["Arrival Minutes"] - hour_midpoint_minutes
    ).abs()

    # Sort by midpoint distance first and scheduled arrival time second.
    # The second key makes the earlier trip win if a tie occurs.
    hour_trips_sorted = hour_trips.sort_values(
        ["Distance to Midpoint", "Arrival Minutes"]
    ).reset_index(drop=True)

    min_distance = hour_trips_sorted[
        "Distance to Midpoint"
    ].iloc[0]

    is_tie = (
        hour_trips_sorted["Distance to Midpoint"] == min_distance
    ).sum() > 1

    removed_trip = hour_trips_sorted.iloc[0]
```

```

removed_records.append({
    "Hour": hour,
    "Period": "Morning" if hour < 12 else "Evening",
    "Baseline Trips": len(hour_trips),
    "Hour Midpoint (minutes)": hour_midpoint_minutes,
    "Removed trip_id": removed_trip["trip_id"],
    "Removed Stop 6 Arrival Time": removed_trip["arrival_time"],
    "Distance to Midpoint (minutes)": round(
        float(min_distance), 3
    ),
    "Tie at Minimum Distance": bool(is_tie)
})

removed_trips_df = pd.DataFrame(removed_records)
removed_trip_ids = set(removed_trips_df["Removed trip_id"])

# Confirm that one unique trip was selected for every study hour.
if len(removed_trip_ids) != len(study_hours):
    raise ValueError(
        "Reduced-Service construction did not select exactly "
        "one unique trip per study hour."
    )

print(
    "Trips removed:",
    len(removed_trip_ids),
    f"(expected: {len(study_hours)})"
)
print(
    "Any ties encountered:",
    removed_trips_df["Tie at Minimum Distance"].any()
)

removed_trips_df

```

Trips removed: 10 (expected: 10)

Any ties encountered: False

Out[11]:

	Hour	Period	Baseline Trips	Hour Midpoint (minutes)	Removed trip_id	Removed Stop 6 Arrival Time	Distance to Midpoint (minutes)	Tie at Minimum Distance
0	6	Morning	5	390	MQ_C6-Weekday-039100_M66_502	06:34:54	4.900	False
1	7	Morning	12	450	MQ_C6-Weekday-044500_M66_509	07:29:54	0.100	False
2	8	Morning	12	510	MQ_C6-Weekday-050600_M66_509	08:31:37	1.617	False
3	9	Morning	8	570	MQ_C6-Weekday-056800_M66_507	09:33:37	3.617	False
4	10	Morning	6	630	MQ_C6-Weekday-062800_M66_513	10:33:48	3.800	False
5	15	Evening	8	930	MQ_C6-Weekday-092300_M66_518	15:28:48	1.200	False
6	16	Evening	12	990	MQ_C6-Weekday-098300_M66_516	16:28:48	1.200	False
7	17	Evening	11	1050	MQ_C6-Weekday-104300_M66_515	17:28:37	1.383	False
8	18	Evening	8	1110	MQ_C6-Weekday-110200_M66_519	18:27:37	2.383	False
9	19	Evening	5	1170	MQ_C6-Weekday-116400_M66_522	19:30:16	0.267	False

In [12]:

```
# Remove the selected trip IDs from all five modeled stops while keeping
# the published GTFS times of every remaining trip unchanged.
reduced_study_stop_times = study_stop_times[
    ~study_stop_times["trip_id"].isin(removed_trip_ids)
].copy()

reduced_first_stop_times = first_stop_times[
    ~first_stop_times["trip_id"].isin(removed_trip_ids)
]
```

```

].copy()

# Count the remaining reference trips in each study hour.
reduced_trips_per_hour = (
    reduced_first_stop_times[
        reduced_first_stop_times["Hour"].isin(study_hours)
    ]
    .groupby("Hour")
    .size()
)

reduced_service_plan = (
    reduced_trips_per_hour
    .reindex(study_hours, fill_value=0)
    .reset_index()
)

reduced_service_plan.columns = ["Hour", "Trips"]

reduced_service_plan["Period"] = np.where(
    reduced_service_plan["Hour"] < 12,
    "Morning",
    "Evening"
)

# Calculate the Morning and Evening service budgets after trip removal.
reduced_service_budget = (
    reduced_service_plan
    .groupby("Period")["Trips"]
    .sum()
    .to_dict()
)

# Compare the hourly reference and Reduced-Service trip counts.
trip_loss_per_hour = (
    baseline_service_plan.set_index("Hour")["Trips"]
    - reduced_service_plan.set_index("Hour")["Trips"]
)

assert (
    trip_loss_per_hour == trips_removed_per_hour

```

```

).all(), (
    "Every study hour must lose exactly one trip "
    "under the Reduced-Service rule."
)

print(
    "Reduced-Service Morning service budget:",
    reduced_service_budget["Morning"]
)
print(
    "Reduced-Service Evening service budget:",
    reduced_service_budget["Evening"]
)
print("Confirmed: every study hour lost exactly one trip.")

reduced_service_plan

```

Reduced-Service Morning service budget: 38
 Reduced-Service Evening service budget: 39
 Confirmed: every study hour lost exactly one trip.

Out[12]:

	Hour	Trips	Period
0	6	4	Morning
1	7	11	Morning
2	8	11	Morning
3	9	7	Morning
4	10	5	Morning
5	15	7	Evening
6	16	11	Evening
7	17	10	Evening
8	18	7	Evening
9	19	4	Evening

```

In [13]: # Create a compact comparison of the Baseline and Reduced-Service
# trip counts for every study hour.
hourly_trip_counts = baseline_service_plan[
    ["Hour", "Period", "Trips"]
].rename(
    columns={"Trips": "Baseline Trips"}
).merge(
    reduced_service_plan[
        ["Hour", "Trips"]
    ].rename(
        columns={"Trips": "Reduced-Service Trips"}
    ),
    on="Hour"
)

hourly_trip_counts["Trips Removed"] = (
    hourly_trip_counts["Baseline Trips"]
    - hourly_trip_counts["Reduced-Service Trips"]
)

# Add a final row showing the total number of reference and remaining trips.
total_trips_row = pd.DataFrame([
    {
        "Hour": "Total",
        "Period": "All",
        "Baseline Trips": int(
            hourly_trip_counts["Baseline Trips"].sum()
        ),
        "Reduced-Service Trips": int(
            hourly_trip_counts["Reduced-Service Trips"].sum()
        ),
        "Trips Removed": int(
            hourly_trip_counts["Trips Removed"].sum()
        )
    }
])

hourly_trip_counts_with_total = pd.concat(
    [hourly_trip_counts, total_trips_row],
    ignore_index=True
)

```

```
hourly_trip_counts_with_total
```

Out[13]:

	Hour	Period	Baseline Trips	Reduced-Service Trips	Trips Removed
0	6	Morning	5	4	1
1	7	Morning	12	11	1
2	8	Morning	12	11	1
3	9	Morning	8	7	1
4	10	Morning	6	5	1
5	15	Evening	8	7	1
6	16	Evening	12	11	1
7	17	Evening	11	10	1
8	18	Evening	8	7	1
9	19	Evening	5	4	1
10	Total	All	87	77	10

6. Validate Reduced-Service Trip Removal

The Reduced-Service construction is validated independently after the removal rule is applied.

The checks confirm that:

- the selected trip is the trip closest to the midpoint of each study hour;
- the earlier trip is selected if an equal-distance tie occurs;
- each removed `trip_id` is removed from all five modeled stops;
- exactly one trip is removed from every study hour; and
- the resulting service budgets are 38 Morning trips and 39 Evening trips.

The notebook stops with an assertion error if any validation check fails.

```
In [14]: # Recompute the expected removed trip independently for every study hour  
# and compare it with the trip selected during schedule construction.  
correct_selection = True  
tie_earlier_correct = True  
  
for hour in study_hours:  
    hour_trips = first_stop_times[  
        first_stop_times["Hour"] == hour  
    ].copy()  
  
    hour_midpoint_minutes = (hour * 60) + 30  
  
    hour_trips["Distance"] = (  
        hour_trips["Arrival Minutes"] - hour_midpoint_minutes  
    ).abs()  
  
    min_distance = hour_trips["Distance"].min()  
  
    tied_candidates = hour_trips[  
        hour_trips["Distance"] == min_distance  
    ].sort_values("Arrival Minutes")  
  
    expected_trip_id = tied_candidates.iloc[0]["trip_id"]  
  
    actual_trip_id = removed_trips_df.loc[  
        removed_trips_df["Hour"] == hour,  
        "Removed trip_id"  
    ].iloc[0]  
  
    if actual_trip_id != expected_trip_id:  
        correct_selection = False  
  
    if (  
        len(tied_candidates) > 1  
        and actual_trip_id != tied_candidates.iloc[0]["trip_id"]  
    ):  
        tie_earlier_correct = False
```

```
# Verify that each selected trip originally contains all five modeled stops  
# and that none of those trip IDs remains in the Reduced-Service schedule.
```

```
original_stop_counts = (  
    study_stop_times[  
        study_stop_times["trip_id"].isin(removed_trip_ids)  
    ]  
    .groupby("trip_id")["stop_sequence"]  
    .nunique()  
)
```

```
removed_trips_cover_all_stops = (  
    len(original_stop_counts) == len(removed_trip_ids)  
    and (original_stop_counts == len(selected_stops)).all()  
)
```

```
no_removed_trips_remain = not reduced_study_stop_times[  
    "trip_id"  
].isin(removed_trip_ids).any()
```

```
full_removal_ok = (  
    removed_trips_cover_all_stops  
    and no_removed_trips_remain  
)
```

```
# Verify the final Reduced-Service trip budgets.
```

```
expected_reduced_budget = {  
    "Morning": (  
        service_budget["Morning"]  
        - trips_removed_per_hour * len(morning_hours)  
    ),  
    "Evening": (  
        service_budget["Evening"]  
        - trips_removed_per_hour * len(evening_hours)  
    )  
}
```

```
service_totals_ok = (  
    reduced_service_budget == expected_reduced_budget  
)
```

```

# Store the validation results in a compact reviewer-facing table.
validation_records = [
    {
        "Check": (
            "Removed trip is closest to the hour midpoint"
        ),
        "Passed": bool(correct_selection)
    },
    {
        "Check": (
            "Earlier scheduled trip is selected when a midpoint tie occurs"
        ),
        "Passed": bool(tie_earlier_correct)
    },
    {
        "Check": (
            "Removed trip IDs are excluded from all five modeled stops"
        ),
        "Passed": bool(full_removal_ok)
    },
    {
        "Check": (
            "Exactly one trip is removed from every study hour"
        ),
        "Passed": bool(
            (
                trip_loss_per_hour
                == trips_removed_per_hour
            ).all()
        )
    },
    {
        "Check": (
            "Reduced-Service period trip budgets are preserved"
        ),
        "Passed": bool(service_totals_ok)
    }
]

```

```
removal_validation_df = pd.DataFrame(validation_records)

assert removal_validation_df["Passed"].all(), (
    "Reduced-Service trip-removal validation failed. "
    "Review removal_validation_df."
)

print(
    "All Reduced-Service trip-removal validation checks passed."
)

removal_validation_df
```

All Reduced-Service trip-removal validation checks passed.

Out[14]:

	Check	Passed
0	Removed trip is closest to the hour midpoint	True
1	Earlier scheduled trip is selected when a midp...	True
2	Removed trip IDs are excluded from all five mo...	True
3	Exactly one trip is removed from every study hour	True
4	Reduced-Service period trip budgets are preserved	True

7. Build Baseline Occupancy and Alighting Inputs

The simulation represents buses as already carrying passengers when they enter the modeled segment at Stop Sequence 6.

Baseline starting occupancy is estimated from ridership activity at Stop Sequences 1 through 5, which are the M66 stops immediately upstream of the five modeled stops. For each study hour, mean upstream boardings and alightings are used to estimate the upstream net passenger flow. This net flow is divided by the number of scheduled buses in that hour and rounded to a whole-passenger starting occupancy, subject to the modeled bus capacity of 60 passengers.

The High-Demand scenario changes only boarding demand at the five modeled stops. It therefore uses the same upstream starting-occupancy inputs as Baseline rather than assuming an unsupported 40% increase in upstream ridership.

Alighting demand at the modeled stops is also derived from the representative stop-hour MTA ridership values. The rounded stop-hour alighting target is distributed across the buses serving that hour using quotient-and-remainder allocation so that the complete stop-hour target is preserved.

```
In [15]: # Load the raw M66 ridership data used to estimate passenger activity
# at the stops immediately upstream of the modeled segment.
raw_m66 = pd.read_csv(raw_file)

# Convert Date and Hour into the formats used for weekday and
# study-period filtering.
raw_m66["Date"] = pd.to_datetime(raw_m66["Date"])
raw_m66["Hour"] = pd.to_datetime(raw_m66["Hour"]).dt.hour

# Identify Stop Sequences 1 through 5 from the same full-route GTFS
# service used by the simulation.
upstream_gtfs_stops = (
    stop_times[
        stop_times["trip_id"].isin(m66_trips["trip_id"])
        & stop_times["stop_sequence"].between(1, 5)
    ][["stop_id", "stop_sequence"]]
    .drop_duplicates()
    .sort_values("stop_sequence")
)

upstream_stop_ids = (
    upstream_gtfs_stops["stop_id"]
    .astype(int)
    .tolist()
)

# Keep weekday ridership records for the upstream stops during the
# Morning and Evening study hours.
upstream_df = raw_m66[
    (raw_m66["Date"].dt.dayofweek < 5)
    & (raw_m66["Stop ID"].isin(upstream_stop_ids))
    & (raw_m66["Hour"].isin(study_hours))
].copy()

# Remove source-inconsistent records where boardings or alightings are
```

```

# reported even though the source record reports zero trips.
upstream_clean = upstream_df[
    ~(
        (upstream_df["Trips"] == 0)
        & (
            (upstream_df["Boardings"] > 0)
            | (upstream_df["Alightings"] > 0)
        )
    )
].copy()

if upstream_clean.empty:
    raise ValueError(
        "No valid upstream ridership records remain after filtering."
    )

print("Verified upstream Stop IDs:", upstream_stop_ids)
print("Upstream rows after cleaning:", len(upstream_clean))

```

Verified upstream Stop IDs: [403487, 403488, 403489, 403491, 403573]
Upstream rows after cleaning: 10867

```

In [16]: # Calculate representative upstream boardings and alightings for each
# upstream stop and study hour.
upstream_stop_hour = (
    upstream_clean
    .groupby(["Stop ID", "Hour"])[["Boardings", "Alightings"]]
    .mean()
    .reset_index()
)

# Sum the representative upstream activity across Stops 1 through 5
# to estimate the total passenger flow entering the modeled segment.
upstream_hourly_flow = (
    upstream_stop_hour
    .groupby("Hour")[["Boardings", "Alightings"]]
    .sum()
    .reset_index()
    .rename(
        columns={
            "Boardings": "Average Upstream Boardings",

```

```

        "Alightings": "Average Upstream Alightings"
    }
)
)

upstream_hourly_flow["Upstream Net Flow"] = (
    upstream_hourly_flow["Average Upstream Boardings"]
    - upstream_hourly_flow["Average Upstream Alightings"]
)

# Match each study hour to the number of reference buses scheduled
# during that hour.
upstream_hourly_flow["Scheduled Buses"] = (
    upstream_hourly_flow["Hour"].map(trips_per_hour)
)

if upstream_hourly_flow["Scheduled Buses"].isna().any():
    raise ValueError(
        "At least one upstream study hour has no matching GTFS service count."
    )

if (upstream_hourly_flow["Scheduled Buses"] <= 0).any():
    raise ValueError(
        "Starting occupancy cannot be estimated for an hour with no buses."
    )

# Estimate the representative starting occupancy per bus from the
# upstream net flow and the number of buses serving the hour.
upstream_hourly_flow["Initial Occupancy Unrounded"] = (
    upstream_hourly_flow["Upstream Net Flow"]
    / upstream_hourly_flow["Scheduled Buses"]
)

# Convert the estimate to a valid whole-passenger occupancy within the
# modeled 60-passenger bus capacity.
upstream_hourly_flow["Initial Occupancy"] = np.minimum(
    bus_capacity,
    np.maximum(
        0,
        np.floor(
            upstream_hourly_flow["Initial Occupancy Unrounded"] + 0.5

```

```
    )
)
).astype(int)

initial_occupancy_by_hour = (
    upstream_hourly_flow
    .set_index("Hour")["Initial Occupancy"]
    .to_dict()
)

upstream_net_flow_by_hour = (
    upstream_hourly_flow
    .set_index("Hour")["Upstream Net Flow"]
    .to_dict()
)

upstream_hourly_flow[
    [
        "Hour",
        "Upstream Net Flow",
        "Scheduled Buses",
        "Initial Occupancy"
    ]
]
```

Out[16]:

	Hour	Upstream Net Flow	Scheduled Buses	Initial Occupancy
0	6	60.132476	5	12
1	7	163.631926	12	14
2	8	211.704836	12	18
3	9	124.995854	8	16
4	10	91.587024	6	15
5	15	137.208668	8	17
6	16	122.268987	12	10
7	17	118.860856	11	11
8	18	95.938313	8	12
9	19	49.363709	5	10

```
In [17]: # Assign each Baseline reference trip the starting occupancy calculated
# for its Stop 6 arrival hour.
trip_initial_occupancy = first_stop_times[
    first_stop_times["Hour"].isin(study_hours)
][["trip_id", "Hour"]].copy()

trip_initial_occupancy["Initial Occupancy"] = (
    trip_initial_occupancy["Hour"].map(initial_occupancy_by_hour)
)

trip_initial_occupancy["Period"] = np.where(
    trip_initial_occupancy["Hour"] < 12,
    "Morning",
    "Evening"
)

if trip_initial_occupancy["Initial Occupancy"].isna().any():
    raise ValueError(
        "At least one Baseline trip has no starting occupancy assignment."
```

```

    )

print(
    "Baseline trips with starting occupancy:",
    len(trip_initial_occupancy)
)

```

Baseline trips with starting occupancy: 87

```

In [18]: # Build the representative whole-passenger alighting target for every
# modeled stop-hour cell.
hourly_alighting_targets = demand_table[
    ["Stop Sequence", "Stop Name", "Hour", "Alightings"]
].copy()

hourly_alighting_targets["Alighting Target"] = np.maximum(
    0,
    np.floor(
        hourly_alighting_targets["Alightings"] + 0.5
    )
).astype(int)

# Distribute each stop-hour alighting target across the buses that
# serve that hour while preserving the complete target.
def build_alighting_plan(
    service_stop_times,
    trip_hour_period_table
):
    """Distribute each stop-hour alighting target across eligible buses while preserving the full integer target."""
    # Start from every eligible bus-stop event that can receive an alighting assignment.
    plan = service_stop_times[
        ["trip_id", "stop_sequence", "arrival_minutes"]
    ].copy()

    # Attach each trip's study hour and period before applying the stop-hour target.
    plan = plan.merge(
        trip_hour_period_table[
            ["trip_id", "Hour", "Period"]
        ],
        on="trip_id",

```

```

        how="left"
    )

    plan = plan.rename(
        columns={"stop_sequence": "Stop Sequence"}
    )

    plan = plan.merge(
        hourly_alighting_targets[
            ["Stop Sequence", "Hour", "Alighting Target"]
        ],
        on=["Stop Sequence", "Hour"],
        how="left"
    )

    if plan["Hour"].isna().any():
        raise ValueError(
            "At least one service trip has no study-hour assignment."
        )

    if plan["Alighting Target"].isna().any():
        raise ValueError(
            "At least one service stop-hour has no alighting target."
        )

    plan = plan.sort_values(
        ["Stop Sequence", "Hour", "arrival_minutes"]
    ).reset_index(drop=True)

    # Add the remainder deterministically so the complete integer target is preserved.
    plan["Assigned Alightings"] = 0

    for (stop, hour), group in plan.groupby(
        ["Stop Sequence", "Hour"]
    ):
        target = int(group["Alighting Target"].iloc[0])
        bus_count = len(group)

        quotient = target // bus_count
        remainder = target % bus_count

```

```

        assignments = [quotient + 1] * remainder
        assignments += [quotient] * (
            bus_count - remainder
        )

        plan.loc[
            group.index,
            "Assigned Alightings"
        ] = assignments

    return plan

# Build the Baseline bus-level alighting assignments and Lookup tables
# used by the discrete-event simulation.
bus_alighting_plan = build_alighting_plan(
    study_stop_times,
    trip_initial_occupancy
)

initial_occupancy_lookup = (
    trip_initial_occupancy
    .set_index("trip_id")["Initial Occupancy"]
    .to_dict()
)

alighting_lookup = (
    bus_alighting_plan
    .set_index(
        ["trip_id", "Stop Sequence"]
    )["Assigned Alightings"]
    .to_dict()
)

trip_period_lookup = (
    trip_initial_occupancy
    .set_index("trip_id")["Period"]
    .to_dict()
)

# Verify that distributing alightings across individual buses preserves

```

```

# every original stop-hour target.
alighting_check = (
    bus_alighting_plan
    .groupby(["Stop Sequence", "Hour"])
    .agg(
        Target=("Alighting Target", "first"),
        Assigned=("Assigned Alightings", "sum")
    )
    .reset_index()
)

baseline_alightings_preserved = (
    alighting_check["Target"]
    == alighting_check["Assigned"]
).all()

assert baseline_alightings_preserved, (
    "Baseline bus-level alighting assignments do not preserve "
    "the stop-hour targets."
)

print(
    "Baseline alighting targets preserved:",
    baseline_alightings_preserved
)

```

Baseline alighting targets preserved: True

8. Build Reduced-Service Occupancy and Alighting Inputs

Reduced-Service retains the same representative upstream passenger flow and the same stop-hour alighting targets as Baseline. However, both inputs must be reassigned because fewer buses serve each study hour.

Starting occupancy is therefore recalculated by dividing the same hourly upstream net flow across the remaining Reduced-Service buses. This generally increases the representative starting occupancy per bus because the same estimated upstream passenger flow is carried by fewer vehicles.

The same Baseline stop-hour alighting targets are then redistributed across the remaining buses using the same quotient-and-remainder procedure.

The Combined scenario uses these Reduced-Service occupancy and alighting inputs because it uses the same reduced service schedule. They are not multiplied by 1.40 because the Combined demand increase applies only to boarding demand at the five modeled stops.

```
In [19]: # Recalculate starting occupancy for each study hour using the same
# upstream net flow but the smaller Reduced-Service bus count.
reduced_initial_occupancy_by_hour = {}

for hour in study_hours:
    bus_count = int(
        reduced_trips_per_hour.get(hour, 0)
    )

    if bus_count <= 0:
        raise ValueError(
            f"Reduced-Service has no bus available in study hour {hour}."
        )

    unrounded_occupancy = (
        upstream_net_flow_by_hour[hour]
        / bus_count
    )

    reduced_initial_occupancy_by_hour[hour] = int(
        np.minimum(
            bus_capacity,
            np.maximum(
                0,
                np.floor(unrounded_occupancy + 0.5)
            )
        )
    )

# Assign the recalculated starting occupancy to every remaining
# Reduced-Service trip.
reduced_trip_initial_occupancy = (
    reduced_first_stop_times[
```

```

        reduced_first_stop_times["Hour"].isin(study_hours)
    ]["trip_id", "Hour"]
    .copy()
)

reduced_trip_initial_occupancy[
    "Initial Occupancy"
] = (
    reduced_trip_initial_occupancy["Hour"]
    .map(reduced_initial_occupancy_by_hour)
)

reduced_trip_initial_occupancy["Period"] = np.where(
    reduced_trip_initial_occupancy["Hour"] < 12,
    "Morning",
    "Evening"
)

if reduced_trip_initial_occupancy[
    "Initial Occupancy"
].isna().any():
    raise ValueError(
        "At least one Reduced-Service trip has no starting "
        "occupancy assignment."
    )

# Build the Reduced-Service occupancy and period lookup tables used
# by the simulation.
reduced_initial_occupancy_lookup = (
    reduced_trip_initial_occupancy
    .set_index("trip_id")["Initial Occupancy"]
    .to_dict()
)

reduced_trip_period_lookup = (
    reduced_trip_initial_occupancy
    .set_index("trip_id")["Period"]
    .to_dict()
)

# Compare Baseline and Reduced-Service starting occupancy by hour.

```

```

occupancy_change = pd.DataFrame({
    "Hour": study_hours,
    "Baseline Initial Occupancy": [
        initial_occupancy_by_hour[hour]
        for hour in study_hours
    ],
    "Reduced-Service Initial Occupancy": [
        reduced_initial_occupancy_by_hour[hour]
        for hour in study_hours
    ]
})

occupancy_change

```

Out[19]:

	Hour	Baseline Initial Occupancy	Reduced-Service Initial Occupancy
--	------	----------------------------	-----------------------------------

0	6	12	15
1	7	14	15
2	8	18	19
3	9	16	18
4	10	15	18
5	15	17	20
6	16	10	11
7	17	11	12
8	18	12	14
9	19	10	12

```

In [20]: # Redistribute the unchanged stop-hour alighting targets across the
# buses that remain in the Reduced-Service schedule.
reduced_bus_alighting_plan = build_alighting_plan(
    reduced_study_stop_times,
    reduced_trip_initial_occupancy
)

```

```

reduced_alighting_lookup = (
    reduced_bus_alighting_plan
    .set_index(
        ["trip_id", "Stop Sequence"]
    )["Assigned Alightings"]
    .to_dict()
)

# Verify that the Reduced-Service bus-level assignments still preserve
# every original stop-hour alighting target.
reduced_alighting_check = (
    reduced_bus_alighting_plan
    .groupby(["Stop Sequence", "Hour"])
    .agg(
        Target=("Alighting Target", "first"),
        Assigned=("Assigned Alightings", "sum")
    )
    .reset_index()
)

reduced_alightings_preserved = (
    reduced_alighting_check["Target"]
    == reduced_alighting_check["Assigned"]
).all()

assert reduced_alightings_preserved, (
    "Reduced-Service bus-level alighting assignments do not "
    "preserve the stop-hour targets."
)

print(
    "Reduced-Service alighting targets preserved:",
    reduced_alightings_preserved
)

```

Reduced-Service alighting targets preserved: True

9. Discrete-Event Simulation Engine

All scenarios use the same discrete-event simulation engine. Scenario differences are supplied through passenger streams, service schedules, starting occupancy, and alighting assignments rather than through separate simulation logic.

Passenger and bus events are processed chronologically. If a bus arrival and passenger arrival occur at exactly the same simulated time, the bus event is processed first.

When a passenger arrives, the passenger joins the FIFO queue at the corresponding stop. When a bus arrives, assigned passengers alight first. The remaining onboard occupancy is then used to determine available capacity, and waiting passengers board in FIFO order up to the modeled 60-passenger capacity.

The simulation records passenger waiting times, queue changes, bus occupancy, completed boardings, and passengers left waiting. Passengers who remain after the last eligible bus are retained as final backlog rather than being served by additional artificial trips.

The same engine produces period-level and stop-level results. It also reports passenger-conservation, waiting-time, and bus-occupancy checks so invalid simulation states can be detected during scenario evaluation.

```
In [21]: # Process passenger arrivals by adding each passenger to the FIFO queue
# at the corresponding modeled stop.
def handle_passenger_arrival(
    event_time,
    stop_sequence,
    passenger_id,
    stop_queues,
    queue_history,
    peak_queue,
    peak_queue_time
):
    """Add one passenger-arrival event to the FIFO queue for the corresponding modeled stop."""
    stop_queues[stop_sequence].append(
        (passenger_id, event_time)
    )

    current_queue = len(stop_queues[stop_sequence])

    # Record the updated queue length after the passenger joins.
    queue_history.append({
        "Time": event_time,
```

```

        "Stop Sequence": stop_sequence,
        "Queue Length": current_queue,
        "Event": "passenger_arrival"
    })

    # Update the stop-level peak queue and the time when it occurs.
    if current_queue > peak_queue[stop_sequence]:
        peak_queue[stop_sequence] = current_queue
        peak_queue_time[stop_sequence] = event_time

# Process each bus arrival using alighting-before-boarding logic and
# enforce the modeled passenger capacity.
def handle_bus_arrival(
    event_time,
    stop_sequence,
    trip_id,
    stop_queues,
    bus_occupancy,
    waiting_records,
    queue_history,
    bus_history,
    occupancy_lookup,
    service_alighting_lookup,
    capacity=bus_capacity
):
    # Initialize the bus occupancy when the trip first enters the
    # modeled five-stop segment.
    """Process one bus arrival using alighting-before-boarding logic and the modeled capacity limit."""
    if trip_id not in bus_occupancy:
        bus_occupancy[trip_id] = int(
            occupancy_lookup[trip_id]
        )

    occupancy_before = bus_occupancy[trip_id]

    # Retrieve the number of passengers assigned to alight at this
    # trip and stop.
    assigned_alightings = int(
        service_alighting_lookup[(trip_id, stop_sequence)]
    )

```

```

# Prevent alightings from exceeding the passengers currently onboard.
actual_alightings = min(
    assigned_alightings,
    bus_occupancy[trip_id]
)

# Apply alightings before calculating the capacity available
# for waiting passengers.
bus_occupancy[trip_id] -= actual_alightings

available_capacity = capacity - bus_occupancy[trip_id]

# Board passengers in FIFO order up to the remaining bus capacity.
boarded = min(
    len(stop_queues[stop_sequence]),
    available_capacity
)

for _ in range(boarded):
    passenger_id, passenger_arrival = stop_queues[
        stop_sequence
    ].pop(0)

    waiting_time = event_time - passenger_arrival

    waiting_records.append({
        "Passenger ID": passenger_id,
        "Stop Sequence": stop_sequence,
        "Trip ID": trip_id,
        "Passenger Arrival": passenger_arrival,
        "Boarding Time": event_time,
        "Waiting Time": waiting_time
    })

    bus_occupancy[trip_id] += boarded

    remaining_queue = len(stop_queues[stop_sequence])

# Record the queue state after the bus finishes boarding.
queue_history.append({

```

```

        "Time": event_time,
        "Stop Sequence": stop_sequence,
        "Queue Length": remaining_queue,
        "Event": "bus_arrival"
    })

    # Record the occupancy and boarding activity for validation
    # and output metrics.
    bus_history.append({
        "Trip ID": trip_id,
        "Stop Sequence": stop_sequence,
        "Time": event_time,
        "Occupancy Before": occupancy_before,
        "Assigned Alightings": assigned_alightings,
        "Actual Alightings": actual_alightings,
        "Boarded": boarded,
        "Occupancy After": bus_occupancy[trip_id],
        "Passengers Left Waiting": remaining_queue
    })

```

```

In [22]: # Build chronological Morning and Evening event queues from the passenger
# stream and service schedule supplied for the current scenario.
#
# Bus events use priority 0 and passenger events use priority 1 so a bus
# is processed first if both events occur at exactly the same time.
def create_period_event_queues(
    passengers,
    service_stop_times,
    trip_period_lookup_table
):
    """Build chronological Morning and Evening event queues for the supplied passenger stream and service plan."""
    morning_events = []
    evening_events = []

    # Add passenger-arrival events to the appropriate study period.
    for _, passenger in passengers.iterrows():
        event = (
            float(passenger["Arrival Minutes"]),
            1,
            "passenger_arrival",

```

```

        int(passenger["Stop Sequence"]),
        passenger["Passenger ID"]
    )

    if passenger["Period"] == "Morning":
        heapq.heappush(morning_events, event)
    else:
        heapq.heappush(evening_events, event)

# Add scheduled bus-arrival events using the service inputs
# supplied for the current scenario.
for _, bus in service_stop_times.iterrows():
    event = (
        float(bus["arrival_minutes"]),
        0,
        "bus_arrival",
        int(bus["stop_sequence"]),
        bus["trip_id"]
    )

    if (
        trip_period_lookup_table[bus["trip_id"]]
        == "Morning"
    ):
        heapq.heappush(morning_events, event)
    else:
        heapq.heappush(evening_events, event)

return morning_events, evening_events

```

```

In [23]: # Run one Morning or Evening simulation period using the shared event
# processing rules and the scenario-specific occupancy and alighting inputs.
#
# The function returns period-level metrics, event histories, stop-level
# results, and validity checks without changing the queueing rules by scenario.
def run_service_simulation(
    period,
    events,
    generated_passengers,
    replication,

```

```

seed,
occupancy_lookup,
service_alighting_lookup
):
    # Define the official boundaries of the requested study period.
    """Run the event-driven queue simulation for one study period and return performance, history, and validation outputs."""
    if period == "Morning":
        start_time = 6 * 60
        end_time = 11 * 60
    else:
        start_time = 15 * 60
        end_time = 20 * 60

    # Copy the event List so the original scenario event queues are
    # not modified during the simulation run.
    simulation_events = events.copy()
    heapq.heapify(simulation_events)

    # Initialize the simulation state for queues, buses, and histories.
    stop_queues = {
        stop: []
        for stop in selected_stops
    }

    bus_occupancy = {}
    waiting_records = []
    queue_history = []
    bus_history = []

    peak_queue = {
        stop: 0
        for stop in selected_stops
    }

    peak_queue_time = {
        stop: None
        for stop in selected_stops
    }

    simulation_clock = start_time
    period_end_queue = None

```

```

period_end_recorded = False

# Record the initial empty queue state at every modeled stop.
for stop in selected_stops:
    queue_history.append({
        "Time": simulation_clock,
        "Stop Sequence": stop,
        "Queue Length": 0,
        "Event": "simulation_start"
    })

# Process passenger and bus events in chronological order.
while simulation_events:
    event = heapq.heappop(simulation_events)

    (
        event_time,
        priority,
        event_type,
        stop_sequence,
        event_id
    ) = event

    # Capture the queue state at the official end of the study
    # period before processing later events.
    if (
        not period_end_recorded
        and event_time > end_time
    ):
        period_end_queue = {
            stop: len(queue)
            for stop, queue in stop_queues.items()
        }

        period_end_recorded = True

    simulation_clock = event_time

    # Apply the appropriate event-processing rule.
    if event_type == "passenger_arrival":
        handle_passenger_arrival(

```

```

        event_time,
        stop_sequence,
        event_id,
        stop_queues,
        queue_history,
        peak_queue,
        peak_queue_time
    )
else:
    handle_bus_arrival(
        event_time,
        stop_sequence,
        event_id,
        stop_queues,
        bus_occupancy,
        waiting_records,
        queue_history,
        bus_history,
        occupancy_lookup,
        service_alighting_lookup,
        capacity=bus_capacity
    )

# If no event occurred after the official period boundary, use the
# final queue state as the recorded period-end queue.
if not period_end_recorded:
    period_end_queue = {
        stop: len(queue)
        for stop, queue in stop_queues.items()
    }

# Passengers still waiting after the final eligible bus form the
# final backlog for the period.
final_backlog = {
    stop: len(queue)
    for stop, queue in stop_queues.items()
}

# Convert recorded simulation histories to DataFrames for metric
# calculation and validation.
waiting_df = pd.DataFrame(waiting_records)

```

```

bus_history_df = pd.DataFrame(bus_history)
queue_history_df = pd.DataFrame(queue_history)

completed_passengers = len(waiting_df)
total_final_backlog = sum(final_backlog.values())

# Verify that every generated passenger is either boarded or remains
# in the final queue.
passenger_conservation = (
    generated_passengers
    == completed_passengers + total_final_backlog
)

# Calculate waiting-time metrics from passengers who successfully board.
mean_wait = waiting_df["Waiting Time"].mean()
median_wait = waiting_df["Waiting Time"].median()
p95_wait = waiting_df["Waiting Time"].quantile(0.95)
max_wait = waiting_df["Waiting Time"].max()

min_wait = (
    waiting_df["Waiting Time"].min()
    if len(waiting_df)
    else np.nan
)

# Verify that no completed passenger has a negative waiting time.
waiting_time_valid = (
    bool(
        (
            waiting_df["Waiting Time"] >= 0
        ).all()
    )
    if len(waiting_df)
    else True
)

# Calculate bus-occupancy limits observed during the simulation.
max_occupancy = int(
    bus_history_df["Occupancy After"].max()
)

```

```

min_occupancy = int(
    min(
        bus_history_df["Occupancy After"].min(),
        bus_history_df["Occupancy Before"].min()
    )
)

# Verify that bus occupancy remains within the modeled capacity
# before and after every bus event.
occupancy_valid = bool(
    bus_history_df[
        "Occupancy After"
    ].between(
        0,
        bus_capacity
    ).all()
    and bus_history_df[
        "Occupancy Before"
    ].between(
        0,
        bus_capacity
    ).all()
)

# Identify the largest queue observed across the five modeled stops.
overall_peak_queue = max(
    peak_queue.values()
)

overall_peak_stop = max(
    peak_queue,
    key=peak_queue.get
)

overall_peak_time = peak_queue_time[
    overall_peak_stop
]

# Store the period-level metrics and validation results.
result = {
    "Replication": int(replication),

```

```
"Seed": int(seed),
"Period": period,
"Generated Passengers": int(
    generated_passengers
),
"Completed Passengers": int(
    completed_passengers
),
"Mean Completed Wait": float(
    mean_wait
),
"Median Completed Wait": float(
    median_wait
),
"P95 Completed Wait": float(
    p95_wait
),
"Maximum Completed Wait": float(
    max_wait
),
"Minimum Completed Wait": float(
    min_wait
),
"Peak Queue": int(
    overall_peak_queue
),
"Peak Queue Stop": int(
    overall_peak_stop
),
"Peak Queue Time": float(
    overall_peak_time
),
"Period-End Queue": int(
    sum(period_end_queue.values())
),
"Final Backlog": int(
    total_final_backlog
),
"Mean Bus Occupancy": float(
    bus_history_df[
        "Occupancy After"
```

```

        ].mean()
    ),
    "Maximum Bus Occupancy": max_occupancy,
    "Minimum Bus Occupancy": min_occupancy,
    "Passenger Conservation": bool(
        passenger_conservation
    ),
    "Waiting Time Valid": waiting_time_valid,
    "Bus Occupancy Valid": occupancy_valid
}

# Calculate the corresponding queue and waiting-time metrics
# separately for each modeled stop.
stop_records = []

for stop in selected_stops:
    stop_waits = (
        waiting_df.loc[
            waiting_df["Stop Sequence"] == stop,
            "Waiting Time"
        ]
        if len(waiting_df)
        else pd.Series(dtype=float)
    )

    stop_records.append({
        "Replication": int(replication),
        "Seed": int(seed),
        "Period": period,
        "Stop Sequence": stop,
        "Completed Passengers": int(
            len(stop_waits)
        ),
        "Mean Wait": (
            float(stop_waits.mean())
            if len(stop_waits)
            else np.nan
        ),
        "P95 Wait": (
            float(
                stop_waits.quantile(0.95)
            )
        )
    })

```

```

        )
        if len(stop_waits)
        else np.nan
    ),
    "Peak Queue": int(
        peak_queue[stop]
    ),
    "Peak Queue Time": (
        peak_queue_time[stop]
    ),
    "Period-End Queue": int(
        period_end_queue[stop]
    ),
    "Final Backlog": int(
        final_backlog[stop]
    )
})

stop_level_df = pd.DataFrame(
    stop_records
)

return (
    result,
    waiting_df,
    bus_history_df,
    queue_history_df,
    stop_level_df
)

```

10. Passenger Stream Generators

Passenger arrivals are generated independently within each stop-hour using a uniform distribution over the corresponding 60-minute interval. The stop-hour passenger counts remain fixed for a scenario; only the within-hour arrival times vary across replications.

Baseline and Reduced-Service use the same Baseline passenger-demand table and the same replication seed. Their passenger streams are therefore identical for a matched replication.

High-Demand preserves the complete Baseline passenger stream and generates only the additional passengers required by the +40% stop-hour demand condition. These additional passengers use a separate deterministic random sub-stream so generating them does not alter the Baseline arrivals.

The Combined scenario uses the same High-Demand passenger generator because its passenger-demand condition is identical to High-Demand.

```
In [24]: # Generate one reproducible passenger stream from a supplied stop-hour
# demand table.
def generate_scenario_passengers(
    demand_df,
    seed,
    replication,
    id_prefix="P",
    id_start=1,
    source_label=None
):
    """Generate a reproducible passenger stream from a supplied stop-hour demand table."""
    rng = np.random.default_rng(seed)
    records = []

    # Generate passenger arrival times independently within each
    # corresponding 60-minute study-hour interval.
    for _, row in demand_df.iterrows():
        start_minute = int(row["Hour"]) * 60
        end_minute = start_minute + 60

        arrival_times = rng.uniform(
            start_minute,
            end_minute,
            int(row["Passenger Count"])
        )

        # Sort arrivals within the stop-hour cell before adding them
        # to the complete passenger stream.
        for arrival_time in sorted(arrival_times):
            records.append({
                "Stop Sequence": int(
                    row["Stop Sequence"]
```

```

    ),
    "Stop Name": row["Stop Name"],
    "Hour": int(row["Hour"]),
    "Period": (
        "Morning"
        if row["Hour"] < 12
        else "Evening"
    ),
    "Arrival Minutes": float(
        arrival_time
    )
})

passengers = pd.DataFrame(records)

# Assign a unique passenger identifier within the replication.
passengers.insert(
    0,
    "Passenger ID",
    [
        f"R{replication:02d}_{id_prefix}{i:05d}"
        for i in range(
            id_start,
            id_start + len(passengers)
        )
    ]
)

# Record the passenger source when a scenario contains more than
# one independently generated component.
if source_label is not None:
    passengers["Passenger Source"] = source_label

return passengers

```

```

In [25]: # Build a High-Demand passenger stream by preserving the complete
# Baseline stream and generating only the additional passengers.
def generate_high_demand_passengers(
    baseline_demand_df,
    additional_demand_df,

```

```

seed,
replication
):
# Generate the Baseline portion using the same demand table and
# replication seed used by the Baseline scenario.
"""Create the High-Demand stream by preserving Baseline passengers and generating only the additional demand."""
baseline_passengers = generate_scenario_passengers(
    baseline_demand_df,
    seed=seed,
    replication=replication,
    id_prefix="P",
    id_start=1,
    source_label="Baseline"
)

# Convert the additional-passenger table into the same input
# structure expected by the general passenger generator.
extra_demand_df = additional_demand_df[
    [
        "Stop Sequence",
        "Stop Name",
        "Hour",
        "Additional Passenger Count"
    ]
].rename(
    columns={
        "Additional Passenger Count":
            "Passenger Count"
    }
)

# Generate only the additional passengers from a separate,
# deterministic random sub-stream.
extra_passengers = generate_scenario_passengers(
    extra_demand_df,
    seed=(seed, 1),
    replication=replication,
    id_prefix="H",
    id_start=1,
    source_label="High-Demand Additional"
)

```

```

# Preserve the complete Baseline stream in its original order and
# append only the additional High-Demand passengers.
return pd.concat(
    [
        baseline_passengers,
        extra_passengers
    ],
    ignore_index=True
)

```

```

In [26]: # Verify that the Baseline portion of a High-Demand replication is
# unchanged when both are generated with the same replication seed.
check_seed = 999999
check_replication = 1

high_demand_check = generate_high_demand_passengers(
    baseline_demand,
    additional_demand,
    check_seed,
    check_replication
)

baseline_check = generate_scenario_passengers(
    baseline_demand,
    check_seed,
    check_replication,
    source_label="Baseline"
)

baseline_portion = (
    high_demand_check[
        high_demand_check["Passenger Source"]
        == "Baseline"
    ]
    .reset_index(drop=True)
)

baseline_stream_matches = np.allclose(
    baseline_portion["Arrival Minutes"].values,

```

```

    baseline_check["Arrival Minutes"].values
)

expected_high_demand_count = (
    len(baseline_check)
    + additional_passenger_total
)

high_demand_count_matches = (
    len(high_demand_check)
    == expected_high_demand_count
)

assert baseline_stream_matches, (
    "The Baseline portion of the High-Demand stream changed."
)

assert high_demand_count_matches, (
    "The High-Demand passenger stream has an unexpected passenger count."
)

print(
    "Baseline portion matches standalone Baseline generation:",
    baseline_stream_matches
)

print(
    "High-Demand passenger count:",
    len(high_demand_check),
    "=",
    len(baseline_check),
    "+",
    additional_passenger_total
)

```

Baseline portion matches standalone Baseline generation:

True

High-Demand passenger count: 1083 = 776 + 307

In [27]: *# Provide scenario-specific passenger generators to the shared
replication runner.*

```

def baseline_generator(seed, replication):
    """Generate the Baseline passenger stream for one replication."""
    return generate_scenario_passengers(
        baseline_demand,
        seed,
        replication,
        id_prefix="P",
        id_start=1,
        source_label="Baseline"
    )

def high_demand_generator(seed, replication):
    """Generate the High-Demand passenger stream for one replication."""
    return generate_high_demand_passengers(
        baseline_demand,
        additional_demand,
        seed,
        replication
    )

def reduced_service_generator(seed, replication):
    """Generate Baseline-demand passengers for a Reduced-Service replication."""
    return generate_scenario_passengers(
        baseline_demand,
        seed,
        replication,
        id_prefix="P",
        id_start=1,
        source_label="Baseline"
    )

```

11. Reproducible Replication Seeds

All scenarios use matched replication seeds generated from the project master seed, 2026 .

A pool of 50 deterministic seeds is created so the simulation can begin with 30 matched replications and, if necessary, extend to 40 or 50 without changing any previously generated replication.

The first 30 generated seeds are checked against the saved Baseline replication-seed file. This ensures that scenario comparisons use the same stochastic passenger-arrival conditions as the established Baseline experiment.

```
In [28]: # Generate a deterministic pool of replication seeds from the project
# master seed.
seed_sequence = np.random.SeedSequence(
    master_seed
)

child_sequences = seed_sequence.spawn(
    precision_max_replications
)

seed_pool = [
    int(child.generate_state(1)[0])
    for child in child_sequences
]

seed_pool_df = pd.DataFrame({
    "Replication": range(
        1,
        precision_max_replications + 1
    ),
    "Seed": seed_pool
})

# Load the saved Baseline seeds used for the original 30-replication
# experiment.
official_baseline_seeds = pd.read_csv(
    processed_dir / "M66_Replication_Seeds.csv"
)

# Verify that the first 30 scenario seeds match the established
# Baseline replication seeds exactly.
baseline_seed_count = len(
    official_baseline_seeds
```

```

)

seeds_match_baseline = np.array_equal(
    seed_pool_df.loc[
        :baseline_seed_count - 1,
        "Seed"
    ].values,
    official_baseline_seeds["Seed"].values
)

assert seeds_match_baseline, (
    "Scenario replication seeds do not match the saved "
    "Baseline seeds."
)

print(
    f"First {baseline_seed_count} scenario seeds "
    "match the Baseline seed file:",
    seeds_match_baseline
)

seed_pool_df.head(10)

```

First 30 scenario seeds match the Baseline seed file: True

Out[28]:

	Replication	Seed
0	1	479243620
1	2	454024514
2	3	1818751028
3	4	2288458700
4	5	2115331525
5	6	2666256175
6	7	975381835
7	8	2535166222
8	9	1079577930
9	10	3715024065

12. Replication Precision Check

Simulation precision is evaluated using a 95% Student's t confidence interval across independent replications.

Precision is checked separately for the Morning and Evening periods using two primary simulation measures:

- Mean Completed Wait
- Peak Queue

For a metric with a replication mean below 10, the required confidence-interval half-width is at most 1.0. For a mean of 10 or greater, the required half-width is at most 10% of the replication mean.

Each scenario begins with 30 replications. If any required precision check fails, additional replications are added in batches of 10 up to a maximum of 50.

```

In [29]: # Evaluate the required 95% confidence-interval precision for the
# primary simulation metrics in both study periods.
def compute_precision_check(results_df):
    """Evaluate the project 95% confidence-interval precision rule for the primary metrics."""
    precision_records = []
    all_passed = True

    for period in ["Morning", "Evening"]:
        period_results = results_df[
            results_df["Period"] == period
        ]

        for metric in [
            "Mean Completed Wait",
            "Peak Queue"
        ]:
            values = period_results[metric]

            n = len(values)

            if n < 2:
                raise ValueError(
                    "At least two replications are required "
                    "for the precision check."
                )

            mean_value = values.mean()
            std_value = values.std(ddof=1)
            standard_error = (
                std_value / np.sqrt(n)
            )

            # Calculate the two-sided Student's t confidence interval.
            t_critical = t.ppf(
                precision_confidence_level
                + (
                    1
                    - precision_confidence_level
                ) / 2,
                df=n - 1

```

```

)

half_width = (
    t_critical
    * standard_error
)

# Apply the project's absolute or relative precision
# threshold according to the replication mean.
if mean_value < 10:
    required_half_width = 1.0
else:
    required_half_width = (
        0.10 * mean_value
    )

passed = (
    half_width
    <= required_half_width
)

all_passed = (
    all_passed
    and passed
)

precision_records.append({
    "Period": period,
    "Metric": metric,
    "Replications": n,
    "Mean": mean_value,
    "Standard Deviation": std_value,
    "CI Lower": (
        mean_value - half_width
    ),
    "CI Upper": (
        mean_value + half_width
    ),
    "Half-Width": half_width,
    "Required Half-Width":
        required_half_width,

```

```

        "Precision Passed": passed
    })

    return (
        pd.DataFrame(
            precision_records
        ),
        all_passed
    )

```

13. Scenario Replication Runner

The shared replication runner executes a complete scenario using its passenger generator, service schedule, starting-occupancy inputs, and alighting assignments.

Each scenario starts with 30 matched replications. After the initial batch, the precision check is applied. If the required precision is not achieved for Mean Completed Wait or Peak Queue in either study period, 10 additional replications are executed. This continues up to the 50-replication limit.

Previously completed replications are retained when the experiment is extended. Only the additional required replications are simulated.

The same runner is used for Baseline, High-Demand, Reduced-Service, and Combined conditions so scenario comparisons use the same simulation and evaluation process.

```

In [30]: # Run a complete scenario with matched replication seeds and extend the
         # experiment only when the required precision has not yet been achieved.
def run_scenario_replications(
    scenario_name,
    passenger_generator_fn,
    service_stop_times,
    trip_period_lookup_table,
    occupancy_lookup_table,
    alighting_lookup_table,
    keep_passenger_streams=False
):
    """Run matched scenario replications and extend the experiment only when the required precision is not met."""
    results = []

```

```

stop_level_records = []
passenger_stream_frames = []

current_target = (
    precision_start_replications
)

completed_reps = 0

while True:
    # Run only the replications that have not already been completed.
    for replication in range(
        completed_reps + 1,
        current_target + 1
    ):
        seed = int(
            seed_pool_df.loc[
                seed_pool_df["Replication"]
                == replication,
                "Seed"
            ].iloc[0]
        )

        passengers = (
            passenger_generator_fn(
                seed,
                replication
            )
        )

        # Retain a Labeled copy of the passenger stream when it is
        # required as an input to downstream optimization.
        if keep_passenger_streams:
            passenger_stream = passengers.copy()

            passenger_stream.insert(
                0,
                "Replication",
                replication
            )

```

```

passenger_stream.insert(
    1,
    "Seed",
    seed
)

passenger_stream_frames.append(
    passenger_stream
)

# Build the Morning and Evening event queues using the
# scenario-specific passenger and service inputs.
(
    morning_events,
    evening_events
) = create_period_event_queues(
    passengers,
    service_stop_times,
    trip_period_lookup_table
)

morning_n = int(
    (
        passengers["Period"]
        == "Morning"
    ).sum()
)

evening_n = int(
    (
        passengers["Period"]
        == "Evening"
    ).sum()
)

# Run the two study periods independently using the same
# replication number and random seed.
(
    morning_result,
    —,
    —,

```

```

        _,
        morning_stop
    ) = run_service_simulation(
        "Morning",
        morning_events,
        morning_n,
        replication,
        seed,
        occupancy_lookup_table,
        alighting_lookup_table
    )

    (
        evening_result,
        _,
        _,
        _,
        evening_stop
    ) = run_service_simulation(
        "Evening",
        evening_events,
        evening_n,
        replication,
        seed,
        occupancy_lookup_table,
        alighting_lookup_table
    )

    results.extend([
        morning_result,
        evening_result
    ])

    stop_level_records.extend([
        morning_stop,
        evening_stop
    ])

    completed_reps = current_target

    # Check whether the completed replication set meets the

```

```

# predefined precision requirement.
results_df = pd.DataFrame(
    results
)

(
    precision_df,
    all_passed
) = compute_precision_check(
    results_df
)

# Stop when precision is achieved or when the maximum
# replication limit has been reached.
if (
    all_passed
    or completed_reps
    >= precision_max_replications
):
    break

current_target = min(
    completed_reps
    + precision_increment,
    precision_max_replications
)

# Add the scenario identifier to the complete result tables.
results_df = pd.DataFrame(
    results
)

results_df.insert(
    0,
    "Scenario",
    scenario_name
)

stop_level_df = pd.concat(
    stop_level_records,
    ignore_index=True
)

```

```

)

stop_level_df.insert(
    0,
    "Scenario",
    scenario_name
)

# Recalculate the final precision table using all replications
# retained for the completed scenario.
(
    precision_df,
    all_passed
) = compute_precision_check(
    results_df
)

precision_df.insert(
    0,
    "Scenario",
    scenario_name
)

# Combine retained passenger streams when requested.
passenger_streams_df = (
    pd.concat(
        passenger_stream_frames,
        ignore_index=True
    )
    if keep_passenger_streams
    else None
)

print(
    f"[{scenario_name}] "
    f"Replications used: {completed_reps} "
    f"(started at "
    f"{precision_start_replications}, "
    f"cap {precision_max_replications}) "
    f"| Precision requirement met: "
    f"{all_passed}"
)

```

```

)

return (
    results_df,
    stop_level_df,
    precision_df,
    passenger_streams_df,
    completed_reps
)

```

14. Run High-Demand Scenario

The High-Demand scenario uses the +40% selected-stop passenger stream together with the unchanged Baseline GTFS service, starting occupancy, and alighting assignments.

The scenario begins with 30 matched replications and may extend to a maximum of 50 if the predefined precision requirement is not satisfied.

```

In [31]: # Run High-Demand using the increased selected-stop passenger demand
# together with the unchanged Baseline service inputs.
(
    high_demand_results_df,
    high_demand_stop_df,
    high_demand_precision_df,
    high_demand_master_streams,
    high_demand_replications_used
) = run_scenario_replications(
    scenario_name="High-Demand",
    passenger_generator_fn=high_demand_generator,
    service_stop_times=study_stop_times,
    trip_period_lookup_table=trip_period_lookup,
    occupancy_lookup_table=initial_occupancy_lookup,
    alighting_lookup_table=alighting_lookup,
    keep_passenger_streams=True
)

# Fail immediately if a core simulation validity condition is violated.
assert high_demand_results_df[
    "Passenger Conservation"

```

```

].all(), (
    "High-Demand failed passenger-conservation validation."
)

assert high_demand_results_df[
    "Waiting Time Valid"
].all(), (
    "High-Demand produced a negative completed waiting time."
)

assert high_demand_results_df[
    "Bus Occupancy Valid"
].all(), (
    "High-Demand produced a bus occupancy outside the valid range."
)

print(
    "Passenger conservation:",
    high_demand_results_df[
        "Passenger Conservation"
    ].all()
)
print(
    "Waiting times valid:",
    high_demand_results_df[
        "Waiting Time Valid"
    ].all()
)
print(
    "Bus occupancy valid:",
    high_demand_results_df[
        "Bus Occupancy Valid"
    ].all()
)

# Verify that the retained High-Demand passenger streams contain the
# replication labels required by the downstream GA notebook.
required_master_stream_columns = {
    "Replication",
    "Seed",

```

```

        "Passenger ID",
        "Stop Sequence",
        "Hour",
        "Period",
        "Arrival Minutes"
    }

    master_stream_columns_ok = (
        required_master_stream_columns
        .issubset(
            high_demand_master_streams.columns
        )
    )

    master_stream_replications_ok = (
        high_demand_master_streams[
            "Replication"
        ].nunique()
        == high_demand_replications_used
    )

    assert master_stream_columns_ok, (
        "High-Demand master passenger streams are missing "
        "columns required by the GA notebook."
    )

    assert master_stream_replications_ok, (
        "High-Demand master passenger streams do not contain "
        "all retained replications."
    )

    print(
        "High-Demand master streams ready for GA:",
        master_stream_columns_ok
        and master_stream_replications_ok
    )

    print(
        "High-Demand master-stream replications:",
        high_demand_master_streams[
            "Replication"

```

```
].unique()  
)
```

```
high_demand_precision_df.round(3)
```

[High-Demand] Replications used: 30 (started at 30, cap 50) | Precision requirement met: True

Passenger conservation: True

Waiting times valid: True

Bus occupancy valid: True

High-Demand master streams ready for GA: True

High-Demand master-stream replications: 30

Out[31]:

	Scenario	Period	Metric	Replications	Mean	Standard Deviation	CI Lower	CI Upper	Half- Width	Required Half- Width	Precision Passed
0	High-Demand	Morning	Mean Completed Wait	30	3.663	0.076	3.634	3.691	0.028	1.000	True
1	High-Demand	Morning	Peak Queue	30	23.367	2.684	22.364	24.369	1.002	2.337	True
2	High-Demand	Evening	Mean Completed Wait	30	3.641	0.131	3.592	3.690	0.049	1.000	True
3	High-Demand	Evening	Peak Queue	30	13.900	2.074	13.126	14.674	0.774	1.390	True

15. Run Reduced-Service Scenario

The Reduced-Service scenario keeps Baseline passenger demand unchanged while using the Reduced-Service reference schedule.

Because fewer buses serve each study hour, the recalculated Reduced-Service starting occupancy and alighting assignments are supplied to the same simulation engine.

The scenario begins with 30 matched replications and may extend to a maximum of 50 if required by the precision rule.

```
In [32]: # Run Reduced-Service using the unchanged Baseline passenger stream  
# and the reduced reference service inputs.  
(
```

```

    reduced_service_results_df,
    reduced_service_stop_df,
    reduced_service_precision_df,
    _,
    reduced_service_replications_used
) = run_scenario_replications(
    scenario_name="Reduced-Service",
    passenger_generator_fn=reduced_service_generator,
    service_stop_times=reduced_study_stop_times,
    trip_period_lookup_table=reduced_trip_period_lookup,
    occupancy_lookup_table=reduced_initial_occupancy_lookup,
    alighting_lookup_table=reduced_alighting_lookup,
    keep_passenger_streams=False
)

# Fail immediately if a core simulation validity condition is violated.
assert reduced_service_results_df[
    "Passenger Conservation"
].all(), (
    "Reduced-Service failed passenger-conservation validation."
)

assert reduced_service_results_df[
    "Waiting Time Valid"
].all(), (
    "Reduced-Service produced a negative completed waiting time."
)

assert reduced_service_results_df[
    "Bus Occupancy Valid"
].all(), (
    "Reduced-Service produced a bus occupancy outside the valid range."
)

print(
    "Passenger conservation:",
    reduced_service_results_df[
        "Passenger Conservation"
    ].all()
)
print(

```

```

    "Waiting times valid:",
    reduced_service_results_df[
        "Waiting Time Valid"
    ].all()
)
print(
    "Bus occupancy valid:",
    reduced_service_results_df[
        "Bus Occupancy Valid"
    ].all()
)

reduced_service_precision_df.round(3)

```

[Reduced-Service] Replications used: 30 (started at 30, cap 50) | Precision requirement met: True
 Passenger conservation: True
 Waiting times valid: True
 Bus occupancy valid: True

Out[32]:

	Scenario	Period	Metric	Replications	Mean	Standard Deviation	CI Lower	CI Upper	Half- Width	Required Half-Width	Precision Passed
0	Reduced-Service	Morning	Mean Completed Wait	30	4.850	0.166	4.788	4.912	0.062	1.000	True
1	Reduced-Service	Morning	Peak Queue	30	24.500	3.026	23.370	25.630	1.130	2.450	True
2	Reduced-Service	Evening	Mean Completed Wait	30	4.368	0.230	4.283	4.454	0.086	1.000	True
3	Reduced-Service	Evening	Peak Queue	30	10.267	1.437	9.730	10.803	0.537	1.027	True

16. Run Combined Scenario

The Combined scenario applies both stress conditions at the same time:

- boarding demand at the five modeled stops is increased by 40%; and

- one published reference trip is removed from every study hour.

The passenger stream is therefore identical to the High-Demand condition, while the service schedule, starting occupancy, and alighting assignments are identical to the Reduced-Service condition.

The upstream occupancy and alighting inputs are not multiplied by 1.40 because the demand increase is defined only for boarding demand at the five modeled stops.

The same simulation engine and matched replication seeds are used as in all other scenarios.

```
In [33]: # Run Combined using the High-Demand passenger stream together with
# the complete Reduced-Service service-side configuration.
(
    combined_results_df,
    combined_stop_df,
    combined_precision_df,
    _,
    combined_replications_used
) = run_scenario_replications(
    scenario_name="Combined",
    passenger_generator_fn=high_demand_generator,
    service_stop_times=reduced_study_stop_times,
    trip_period_lookup_table=reduced_trip_period_lookup,
    occupancy_lookup_table=reduced_initial_occupancy_lookup,
    alighting_lookup_table=reduced_alighting_lookup,
    keep_passenger_streams=False
)

# Fail immediately if a core simulation validity condition is violated.
assert combined_results_df[
    "Passenger Conservation"
].all(), (
    "Combined failed passenger-conservation validation."
)

assert combined_results_df[
    "Waiting Time Valid"
].all(), (
    "Combined produced a negative completed waiting time."
```

```

)

assert combined_results_df[
    "Bus Occupancy Valid"
].all(), (
    "Combined produced a bus occupancy outside the valid range."
)

print(
    "Passenger conservation:",
    combined_results_df[
        "Passenger Conservation"
    ].all()
)
print(
    "Waiting times valid:",
    combined_results_df[
        "Waiting Time Valid"
    ].all()
)
print(
    "Bus occupancy valid:",
    combined_results_df[
        "Bus Occupancy Valid"
    ].all()
)

combined_precision_df.round(3)

```

[Combined] Replications used: 30 (started at 30, cap 50) | Precision requirement met: True

Passenger conservation: True

Waiting times valid: True

Bus occupancy valid: True

Out[33]:

	Scenario	Period	Metric	Replications	Mean	Standard Deviation	CI Lower	CI Upper	Half-Width	Required Half-Width	Precision Passed
0	Combined	Morning	Mean Completed Wait	30	4.817	0.153	4.760	4.874	0.057	1.00	True
1	Combined	Morning	Peak Queue	30	32.900	4.302	31.294	34.506	1.606	3.29	True
2	Combined	Evening	Mean Completed Wait	30	4.331	0.176	4.265	4.397	0.066	1.00	True
3	Combined	Evening	Peak Queue	30	14.400	1.812	13.723	15.077	0.677	1.44	True

17. Run Baseline Reference for Comparison

The Baseline condition is rerun through the same shared scenario runner used by the three stress scenarios.

This provides a directly comparable reference using the same replication-seed pool, simulation engine, precision rule, service inputs, and output structure used in the current scenario experiment.

```
In [34]: # Rerun Baseline through the same scenario runner used for all
# stress conditions so the resulting tables have a common structure.
(
    baseline_results_df,
    baseline_stop_df,
    baseline_precision_df,
    _,
    baseline_replications_used
) = run_scenario_replications(
    scenario_name="Baseline",
    passenger_generator_fn=baseline_generator,
    service_stop_times=study_stop_times,
    trip_period_lookup_table=trip_period_lookup,
    occupancy_lookup_table=initial_occupancy_lookup,
    alighting_lookup_table=alighting_lookup,
    keep_passenger_streams=False
)
```

```
# Fail immediately if a core simulation validity condition is violated.
assert baseline_results_df[
    "Passenger Conservation"
].all(), (
    "Baseline failed passenger-conservation validation."
)

assert baseline_results_df[
    "Waiting Time Valid"
].all(), (
    "Baseline produced a negative completed waiting time."
)

assert baseline_results_df[
    "Bus Occupancy Valid"
].all(), (
    "Baseline produced a bus occupancy outside the valid range."
)

print(
    "Passenger conservation:",
    baseline_results_df[
        "Passenger Conservation"
    ].all()
)
print(
    "Waiting times valid:",
    baseline_results_df[
        "Waiting Time Valid"
    ].all()
)
print(
    "Bus occupancy valid:",
    baseline_results_df[
        "Bus Occupancy Valid"
    ].all()
)

baseline_precision_df.round(3)
```

[Baseline] Replications used: 30 (started at 30, cap 50) | Precision requirement met: True
Passenger conservation: True
Waiting times valid: True
Bus occupancy valid: True

Out[34]:

	Scenario	Period	Metric	Replications	Mean	Standard Deviation	CI Lower	CI Upper	Half- Width	Required Half- Width	Precision Passed
0	Baseline	Morning	Mean Completed Wait	30	3.657	0.110	3.616	3.698	0.041	1.000	True
1	Baseline	Morning	Peak Queue	30	17.267	1.999	16.520	18.013	0.746	1.727	True
2	Baseline	Evening	Mean Completed Wait	30	3.651	0.164	3.590	3.712	0.061	1.000	True
3	Baseline	Evening	Peak Queue	30	9.867	1.570	9.280	10.453	0.586	1.000	True

18. Validate Simulation Results

Validation is consolidated across Baseline, High-Demand, Reduced-Service, and Combined before the scenario results are summarized or interpreted.

For each scenario, the checks confirm that:

- every generated passenger is accounted for as either successfully boarded or remaining in final backlog;
- no completed passenger has a negative waiting time;
- bus occupancy remains between zero and the modeled 60-passenger capacity;
- the required 95% confidence-interval precision is satisfied; and
- every executed replication uses the corresponding deterministic seed from the common project seed pool.

The maximum observed bus occupancy is also reported so capacity behavior can be inspected directly rather than relying only on a Boolean validity flag.

The notebook stops if any required validation condition fails.

```

In [35]: # Collect the result and precision tables for all four simulation
# conditions in one structure for consolidated validation.
scenario_validation_inputs = {
    "Baseline": {
        "results": baseline_results_df,
        "precision": baseline_precision_df
    },
    "High-Demand": {
        "results": high_demand_results_df,
        "precision": high_demand_precision_df
    },
    "Reduced-Service": {
        "results": reduced_service_results_df,
        "precision": reduced_service_precision_df
    },
    "Combined": {
        "results": combined_results_df,
        "precision": combined_precision_df
    }
}

validation_records = []

# Evaluate the same core simulation and precision checks for every
# scenario using the complete set of retained replications.
for scenario_name, scenario_data in scenario_validation_inputs.items():
    results_df = scenario_data["results"]
    precision_df = scenario_data["precision"]

    replications_used = int(
        results_df["Replication"].nunique()
    )

    passenger_conservation_ok = bool(
        results_df["Passenger Conservation"].all()
    )

    waiting_time_ok = bool(
        results_df["Waiting Time Valid"].all()
    )

```

```

bus_occupancy_ok = bool(
    results_df["Bus Occupancy Valid"].all()
)

precision_ok = bool(
    precision_df["Precision Passed"].all()
)

max_observed_occupancy = int(
    results_df["Maximum Bus Occupancy"].max()
)

validation_records.append({
    "Scenario": scenario_name,
    "Replications Used": replications_used,
    "Passenger Conservation": passenger_conservation_ok,
    "Waiting Time Valid": waiting_time_ok,
    "Bus Occupancy Valid": bus_occupancy_ok,
    "Precision Passed": precision_ok,
    "Maximum Observed Bus Occupancy": max_observed_occupancy,
    "Modeled Bus Capacity": bus_capacity
})

validation_summary_df = pd.DataFrame(
    validation_records
)

# Mark a scenario as fully valid only when every required validation
# and precision condition passes.
validation_summary_df["All Required Checks Passed"] = (
    validation_summary_df[
        [
            "Passenger Conservation",
            "Waiting Time Valid",
            "Bus Occupancy Valid",
            "Precision Passed"
        ]
    ]
    .all(axis=1)
)

```

```

assert validation_summary_df[
    "All Required Checks Passed"
].all(), (
    "At least one scenario failed a required simulation "
    "or precision validation check."
)

print(
    "All required validation checks passed:",
    validation_summary_df[
        "All Required Checks Passed"
    ].all()
)

validation_summary_df

```

All required validation checks passed: True

Out[35]:

	Scenario	Replications Used	Passenger Conservation	Waiting Time Valid	Bus Occupancy Valid	Precision Passed	Maximum Observed Bus Occupancy	Modeled Bus Capacity	All Required Checks Passed
0	Baseline	30	True	True	True	True	33	60	True
1	High-Demand	30	True	True	True	True	45	60	True
2	Reduced-Service	30	True	True	True	True	52	60	True
3	Combined	30	True	True	True	True	60	60	True

```

In [36]: # Verify that every executed replication uses the deterministic seed
# assigned to that replication in the common project seed pool.
matched_seed_records = []

for scenario_name, scenario_data in scenario_validation_inputs.items():
    scenario_seed_map = (
        scenario_data["results"][
            ["Replication", "Seed"]

```

```

    ]
    .drop_duplicates()
    .sort_values("Replication")
    .reset_index(drop=True)
)

expected_seed_map = (
    seed_pool_df[
        seed_pool_df["Replication"].isin(
            scenario_seed_map["Replication"]
        )
    ]
    .sort_values("Replication")
    .reset_index(drop=True)
    .rename(
        columns={
            "Seed": "Expected Seed"
        }
    )
)

seed_check = scenario_seed_map.merge(
    expected_seed_map,
    on="Replication",
    how="left"
)

seeds_match = bool(
    seed_check["Expected Seed"].notna().all()
    and (
        seed_check["Seed"]
        == seed_check["Expected Seed"]
    ).all()
)

matched_seed_records.append({
    "Scenario": scenario_name,
    "Replications Used": len(
        scenario_seed_map
    ),
    "Matched Common Seed Pool": seeds_match
})

```

```

    })

    matched_seed_validation_df = pd.DataFrame(
        matched_seed_records
    )

    assert matched_seed_validation_df[
        "Matched Common Seed Pool"
    ].all(), (
        "At least one scenario does not use the expected "
        "deterministic replication seeds."
    )

    print(
        "All scenarios use the common replication seed pool:",
        matched_seed_validation_df[
            "Matched Common Seed Pool"
        ].all()
    )

    matched_seed_validation_df

```

All scenarios use the common replication seed pool: True

Out[36]:

	Scenario	Replications Used	Matched Common Seed Pool
0	Baseline	30	True
1	High-Demand	30	True
2	Reduced-Service	30	True
3	Combined	30	True

In [37]:

```

# Calculate the total generated passenger count within each complete
# replication so the executed scenarios can be checked against their
# intended demand conditions.
scenario_passenger_records = []

for scenario_name, scenario_data in scenario_validation_inputs.items():
    results_df = scenario_data["results"]

```

```

generated_per_replication = (
    results_df
    .groupby("Replication")[
        "Generated Passengers"
    ]
    .sum()
)

consistent_count = (
    generated_per_replication.nunique()
    == 1
)

passenger_count = int(
    generated_per_replication.iloc[0]
)

scenario_passenger_records.append({
    "Scenario": scenario_name,
    "Passengers per Replication": passenger_count,
    "Consistent Across Replications": consistent_count
})

scenario_passenger_validation_df = pd.DataFrame(
    scenario_passenger_records
)

# Check the expected passenger-demand relationship between scenarios.
passenger_count_lookup = (
    scenario_passenger_validation_df
    .set_index("Scenario")[
        "Passengers per Replication"
    ]
    .to_dict()
)

demand_definition_ok = (
    passenger_count_lookup["Baseline"]
    == baseline_total
    and passenger_count_lookup["Reduced-Service"]

```

```

    == baseline_total
    and passenger_count_lookup["High-Demand"]
    == high_demand_total
    and passenger_count_lookup["Combined"]
    == high_demand_total
    and scenario_passenger_validation_df[
        "Consistent Across Replications"
    ].all()
)

assert demand_definition_ok, (
    "Executed scenario passenger counts do not match "
    "the defined demand conditions."
)

print(
    "Scenario passenger-demand definitions preserved:",
    demand_definition_ok
)

scenario_passenger_validation_df

```

Scenario passenger-demand definitions preserved: True

Out[37]:

	Scenario	Passengers per Replication	Consistent Across Replications
0	Baseline	776	True
1	High-Demand	1083	True
2	Reduced-Service	776	True
3	Combined	1083	True

19. Scenario Configuration

The four simulation conditions differ only through their defined passenger-demand and service inputs. All scenarios use the same five modeled stops, Morning and Evening study periods, simulation engine, 60-passenger modeled bus capacity, and matched replication-seed structure.

Baseline and High-Demand use the published GTFS reference service with 43 Morning trips and 44 Evening trips. Reduced-Service and Combined remove one reference trip from each study hour, producing 38 Morning trips and 39 Evening trips.

Baseline and Reduced-Service use the Baseline selected-stop passenger demand of 776 passengers per replication. High-Demand and Combined use the increased selected-stop demand of 1,083 passengers per replication after stop-hour whole-passenger rounding.

```
In [38]: # Summarize the demand, service, and replication configuration used
# by each of the four executed simulation conditions.
scenario_config_df = pd.DataFrame([
    {
        "Scenario": "Baseline",
        "Demand Condition": "Baseline selected-stop demand",
        "Demand Multiplier": 1.00,
        "Passenger Total per Replication": baseline_total,
        "Service Condition": "Published GTFS reference",
        "Trips Removed per Hour": 0,
        "Morning Trip Budget": service_budget["Morning"],
        "Evening Trip Budget": service_budget["Evening"],
        "Replications Used": baseline_replications_used,
        "Modeled Bus Capacity": bus_capacity
    },
    {
        "Scenario": "High-Demand",
        "Demand Condition": "+40% selected-stop demand",
        "Demand Multiplier": demand_growth_factor,
        "Passenger Total per Replication": high_demand_total,
        "Service Condition": "Published GTFS reference",
        "Trips Removed per Hour": 0,
        "Morning Trip Budget": service_budget["Morning"],
        "Evening Trip Budget": service_budget["Evening"],
        "Replications Used": high_demand_replications_used,
        "Modeled Bus Capacity": bus_capacity
    },
    {
        "Scenario": "Reduced-Service",
        "Demand Condition": "Baseline selected-stop demand",
        "Demand Multiplier": 1.00,
        "Passenger Total per Replication": baseline_total,
        "Service Condition": "Reduced-Service reference",
```

```

        "Trips Removed per Hour": trips_removed_per_hour,
        "Morning Trip Budget": reduced_service_budget["Morning"],
        "Evening Trip Budget": reduced_service_budget["Evening"],
        "Replications Used": reduced_service_replications_used,
        "Modeled Bus Capacity": bus_capacity
    },
    {
        "Scenario": "Combined",
        "Demand Condition": "+40% selected-stop demand",
        "Demand Multiplier": demand_growth_factor,
        "Passenger Total per Replication": high_demand_total,
        "Service Condition": "Reduced-Service reference",
        "Trips Removed per Hour": trips_removed_per_hour,
        "Morning Trip Budget": reduced_service_budget["Morning"],
        "Evening Trip Budget": reduced_service_budget["Evening"],
        "Replications Used": combined_replications_used,
        "Modeled Bus Capacity": bus_capacity
    }
])

# Calculate the complete Morning-and-Evening trip budget for each
# scenario from its period-level service budgets.
scenario_config_df["Total Trip Budget"] = (
    scenario_config_df["Morning Trip Budget"]
    + scenario_config_df["Evening Trip Budget"]
)

scenario_config_df

```

Out[38]:

	Scenario	Demand Condition	Demand Multiplier	Passenger Total per Replication	Service Condition	Trips Removed per Hour	Morning Trip Budget	Evening Trip Budget	Replications Used	Modeled Bus Capacity	Total Trip Budget
0	Baseline	Baseline selected-stop demand	1.0	776	Published GTFS reference	0	43	44	30	60	87
1	High-Demand	+40% selected-stop demand	1.4	1083	Published GTFS reference	0	43	44	30	60	87
2	Reduced-Service	Baseline selected-stop demand	1.0	776	Reduced-Service reference	1	38	39	30	60	77
3	Combined	+40% selected-stop demand	1.4	1083	Reduced-Service reference	1	38	39	30	60	77

In [39]: *# Verify that the configuration table matches the demand, service, # and replication counts of the executed scenario runs.*

```
executed_replication_counts = {
    "Baseline": int(
        baseline_results_df[
            "Replication"
        ].nunique()
    ),
    "High-Demand": int(
        high_demand_results_df[
            "Replication"
        ].nunique()
    ),
    "Reduced-Service": int(
        reduced_service_results_df[
            "Replication"
        ].nunique()
    )
}
```

```

        ].nunique()
    ),
    "Combined": int(
        combined_results_df[
            "Replication"
        ].nunique()
    )
}

configured_replication_counts = (
    scenario_config_df
    .set_index("Scenario")[
        "Replications Used"
    ]
    .to_dict()
)

replication_counts_match = all(
    int(
        configured_replication_counts[
            scenario_name
        ]
    )
    == executed_replications
    for (
        scenario_name,
        executed_replications
    ) in executed_replication_counts.items()
)

configuration_checks = {
    "Baseline passenger total": (
        scenario_config_df.loc[
            scenario_config_df["Scenario"]
            == "Baseline",
            "Passenger Total per Replication"
        ].iloc[0]
        == 776
    ),
    "High-Demand passenger total": (
        scenario_config_df.loc[

```

```

        scenario_config_df["Scenario"]
        == "High-Demand",
        "Passenger Total per Replication"
    ].iloc[0]
    == 1083
),
"Reduced-Service trip budget": (
    scenario_config_df.loc[
        scenario_config_df["Scenario"]
        == "Reduced-Service",
        "Total Trip Budget"
    ].iloc[0]
    == 77
),
"Combined trip budget": (
    scenario_config_df.loc[
        scenario_config_df["Scenario"]
        == "Combined",
        "Total Trip Budget"
    ].iloc[0]
    == 77
),
"Replication counts match executed results":
    replication_counts_match
}

configuration_validation_df = pd.DataFrame({
    "Check": configuration_checks.keys(),
    "Passed": configuration_checks.values()
})

assert configuration_validation_df[
    "Passed"
].all(), (
    "Scenario configuration does not match "
    "the executed experiment."
)

print(
    "Scenario configuration matches executed experiment:",
    configuration_validation_df[

```

```
    "Passed"  
    ].all()  
)
```

```
configuration_validation_df
```

Scenario configuration matches executed experiment: True

Out[39]:

	Check	Passed
0	Baseline passenger total	True
1	High-Demand passenger total	True
2	Reduced-Service trip budget	True
3	Combined trip budget	True
4	Replication counts match executed results	True

20. Scenario Comparison Summary

The completed Baseline, High-Demand, Reduced-Service, and Combined simulation results are summarized by study period.

Each reported value is based on the replications retained after the precision check. The summary includes the primary waiting-time and queue measures together with final backlog and bus-occupancy measures.

For metrics that vary by replication, the table reports the mean across replications. **Overall Maximum Observed Occupancy** is reported separately as the highest bus occupancy recorded in any retained replication for that scenario and period.

These results are descriptive scenario outcomes. Interpretation of differences is performed only after the complete output and validation tables are confirmed.

```
In [40]: # Combine the completed result tables so all four scenarios can be  
# summarized using the same aggregation procedure.  
all_scenario_results = pd.concat(  
    [  
        baseline_results_df,
```

```

        high_demand_results_df,
        reduced_service_results_df,
        combined_results_df
    ],
    ignore_index=True
)

scenario_order = [
    "Baseline",
    "High-Demand",
    "Reduced-Service",
    "Combined"
]

period_order = [
    "Morning",
    "Evening"
]

# Summarize waiting time, queue, backlog, and occupancy measures across
# the retained replications for each scenario and study period.
scenario_summary_df = (
    all_scenario_results
    .groupby(
        ["Scenario", "Period"]
    )
    .agg(
        Replications=(
            "Replication",
            "nunique"
        ),
        **{
            "Mean Completed Wait (minutes)": (
                "Mean Completed Wait",
                "mean"
            ),
            "Mean P95 Completed Wait (minutes)": (
                "P95 Completed Wait",
                "mean"
            ),
            "Mean Peak Queue (passengers)": (

```

```

        "Peak Queue",
        "mean"
    ),
    "Mean Period-End Queue (passengers)": (
        "Period-End Queue",
        "mean"
    ),
    "Mean Final Backlog (passengers)": (
        "Final Backlog",
        "mean"
    ),
    "Mean Bus Occupancy (passengers)": (
        "Mean Bus Occupancy",
        "mean"
    ),
    "Mean Replication Maximum Occupancy": (
        "Maximum Bus Occupancy",
        "mean"
    ),
    "Overall Maximum Observed Occupancy": (
        "Maximum Bus Occupancy",
        "max"
    )
    )
}

)
.reset_index()

)

# Arrange the summary in the same scenario and period order used
# throughout the notebook.
scenario_summary_df["Scenario"] = pd.Categorical(
    scenario_summary_df["Scenario"],
    categories=scenario_order,
    ordered=True
)

scenario_summary_df["Period"] = pd.Categorical(
    scenario_summary_df["Period"],
    categories=period_order,
    ordered=True
)

```

```

scenario_summary_df = (
    scenario_summary_df
    .sort_values(
        ["Scenario", "Period"]
    )
    .reset_index(drop=True)
)

# Confirm that all four scenarios and both study periods are represented.
expected_summary_rows = (
    len(scenario_order)
    * len(period_order)
)

assert len(scenario_summary_df) == expected_summary_rows, (
    "Scenario summary does not contain all scenario-period combinations."
)

# Confirm that each scenario-period summary contains the number of
# replications actually retained by that scenario.
expected_replications_by_scenario = {
    "Baseline": baseline_replications_used,
    "High-Demand": high_demand_replications_used,
    "Reduced-Service": reduced_service_replications_used,
    "Combined": combined_replications_used
}

summary_replication_counts_ok = all(
    (
        scenario_summary_df.loc[
            scenario_summary_df["Scenario"]
            == scenario_name,
            "Replications"
        ]
        == expected_replications
    ).all()
    for (
        scenario_name,
        expected_replications
    ) in expected_replications_by_scenario.items()
)

```

```
)

assert summary_replication_counts_ok, (
    "At least one scenario-period summary contains "
    "an unexpected replication count."
)

scenario_summary_df.round(3)
```

Out[40]:

	Scenario	Period	Replications	Mean Completed Wait (minutes)	Mean P95 Completed Wait (minutes)	Mean Peak Queue (passengers)	Mean Period-End Queue (passengers)	Mean Final Backlog (passengers)	Mean Bus Occupancy (passengers)	Mean Replication Maximum Occupancy	Over Maximum Observed Occupancy
0	Baseline	Morning	30	3.657	9.081	17.267	3.600	2.167	15.922	29.400	3
1	Baseline	Evening	30	3.651	7.823	9.867	3.233	3.233	12.217	24.000	2
2	High-Demand	Morning	30	3.663	9.038	23.367	5.033	2.867	17.924	37.267	4
3	High-Demand	Evening	30	3.641	7.832	13.900	4.267	4.267	13.829	28.633	3
4	Reduced-Service	Morning	30	4.850	14.386	24.500	3.600	2.167	17.701	42.900	5
5	Reduced-Service	Evening	30	4.368	11.917	10.267	3.233	3.233	13.885	27.900	3
6	Combined	Morning	30	4.817	14.290	32.900	5.033	2.867	19.967	54.933	6
7	Combined	Evening	30	4.331	11.756	14.400	4.267	4.267	15.705	32.167	4

```
In [41]: # Create a descriptive comparison of each stress scenario against the
# corresponding Baseline study-period result.
baseline_summary = (
    scenario_summary_df[
        scenario_summary_df["Scenario"]
        == "Baseline"
    ]
```

```

    ]
    [
        "Period",
        "Mean Completed Wait (minutes)",
        "Mean P95 Completed Wait (minutes)",
        "Mean Peak Queue (passengers)",
        "Mean Period-End Queue (passengers)",
        "Mean Final Backlog (passengers)",
        "Mean Bus Occupancy (passengers)"
    ]
    ].copy()
)

baseline_summary = baseline_summary.rename(
    columns={
        "Mean Completed Wait (minutes)":
            "Baseline Mean Wait",
        "Mean P95 Completed Wait (minutes)":
            "Baseline Mean P95 Wait",
        "Mean Peak Queue (passengers)":
            "Baseline Mean Peak Queue",
        "Mean Period-End Queue (passengers)":
            "Baseline Mean Period-End Queue",
        "Mean Final Backlog (passengers)":
            "Baseline Mean Final Backlog",
        "Mean Bus Occupancy (passengers)":
            "Baseline Mean Bus Occupancy"
    }
)

stress_summary = (
    scenario_summary_df[
        scenario_summary_df["Scenario"]
            != "Baseline"
    ]
    ).copy()
)

scenario_change_df = stress_summary.merge(
    baseline_summary,

```

```

    on="Period",
    how="left"
)

# Calculate descriptive differences from the matching Baseline period.
# Positive values indicate a higher scenario value than Baseline.
scenario_change_df[
    "Difference in Mean Wait"
] = (
    scenario_change_df[
        "Mean Completed Wait (minutes)"
    ]
    - scenario_change_df[
        "Baseline Mean Wait"
    ]
)

scenario_change_df[
    "Difference in Mean P95 Wait"
] = (
    scenario_change_df[
        "Mean P95 Completed Wait (minutes)"
    ]
    - scenario_change_df[
        "Baseline Mean P95 Wait"
    ]
)

scenario_change_df[
    "Difference in Mean Peak Queue"
] = (
    scenario_change_df[
        "Mean Peak Queue (passengers)"
    ]
    - scenario_change_df[
        "Baseline Mean Peak Queue"
    ]
)

scenario_change_df[
    "Difference in Mean Period-End Queue"

```

```

] = (
    scenario_change_df[
        "Mean Period-End Queue (passengers)"
    ]
    - scenario_change_df[
        "Baseline Mean Period-End Queue"
    ]
)

scenario_change_df[
    "Difference in Mean Final Backlog"
] = (
    scenario_change_df[
        "Mean Final Backlog (passengers)"
    ]
    - scenario_change_df[
        "Baseline Mean Final Backlog"
    ]
)

scenario_change_df[
    "Difference in Mean Bus Occupancy"
] = (
    scenario_change_df[
        "Mean Bus Occupancy (passengers)"
    ]
    - scenario_change_df[
        "Baseline Mean Bus Occupancy"
    ]
)

scenario_change_df[
    [
        "Scenario",
        "Period",
        "Difference in Mean Wait",
        "Difference in Mean P95 Wait",
        "Difference in Mean Peak Queue",
        "Difference in Mean Period-End Queue",
        "Difference in Mean Final Backlog",
        "Difference in Mean Bus Occupancy"
    ]

```

```
]
].round(3)
```

Out[41]:

	Scenario	Period	Difference in Mean Wait	Difference in Mean P95 Wait	Difference in Mean Peak Queue	Difference in Mean Period-End Queue	Difference in Mean Final Backlog	Difference in Mean Bus Occupancy
0	High-Demand	Morning	0.005	-0.043	6.100	1.433	0.700	2.002
1	High-Demand	Evening	-0.010	0.009	4.033	1.033	1.033	1.612
2	Reduced-Service	Morning	1.193	5.306	7.233	0.000	0.000	1.779
3	Reduced-Service	Evening	0.717	4.093	0.400	0.000	0.000	1.669
4	Combined	Morning	1.160	5.209	15.633	1.433	0.700	4.045
5	Combined	Evening	0.680	3.932	4.533	1.033	1.033	3.488

In [42]: *# Plot three primary scenario measures for Morning and Evening so the
four operating conditions can be compared visually.*

```
comparison_metrics = [
    (
        "Mean Completed Wait (minutes)",
        "Mean completed wait (minutes)"
    ),
    (
        "Mean Peak Queue (passengers)",
        "Mean peak queue (passengers)"
    ),
    (
        "Mean Final Backlog (passengers)",
        "Mean final backlog (passengers)"
    )
]

fig, axes = plt.subplots(
```

```

1,
3,
figsize=(16, 4.5)
)

for ax, (
    metric,
    ylabel
) in zip(
    axes,
    comparison_metrics
):
    pivot = scenario_summary_df.pivot(
        index="Period",
        columns="Scenario",
        values=metric
    )

    pivot = pivot.reindex(
        index=period_order,
        columns=scenario_order
    )

    pivot.plot(
        kind="bar",
        ax=ax,
        legend=False
    )

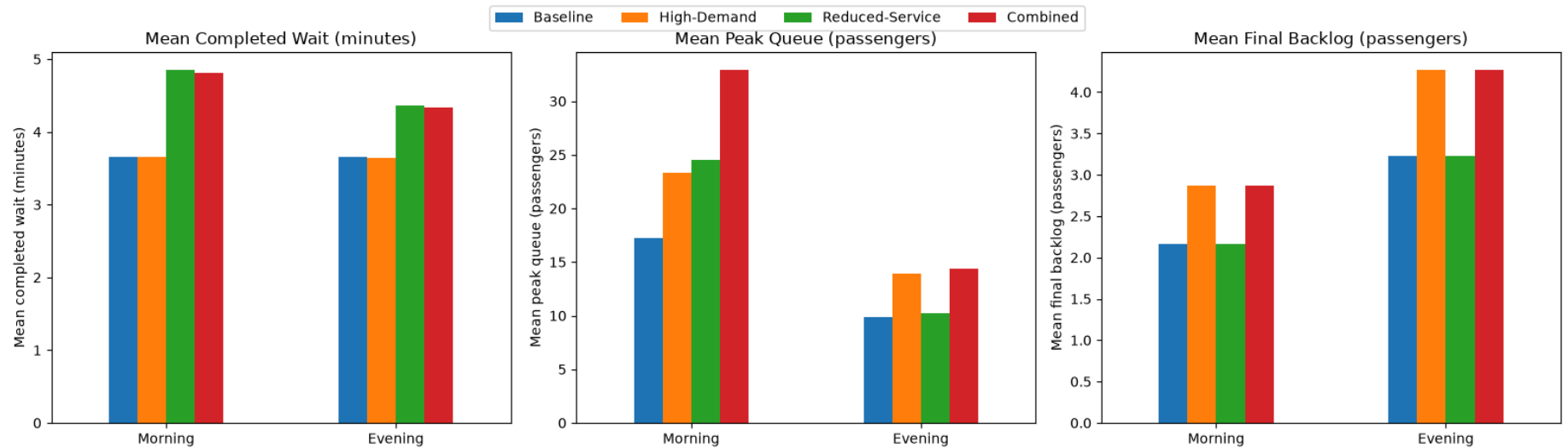
    ax.set_title(metric)
    ax.set_ylabel(ylabel)
    ax.set_xlabel("")
    ax.tick_params(
        axis="x",
        rotation=0
    )

# Use one shared legend for the complete figure.
handles, labels = axes[0].get_legend_handles_labels()

fig.legend(

```

```
handles,  
labels,  
loc="upper center",  
ncol=4,  
bbox_to_anchor=(0.5, 1.04)  
)  
  
fig.tight_layout()  
  
comparison_figure_path = (  
    figures_dir  
    / "scenario_comparison_summary.png"  
)  
  
fig.savefig(  
    comparison_figure_path,  
    dpi=300,  
    bbox_inches="tight"  
)  
  
plt.show()  
  
print(  
    "Saved scenario comparison figure:",  
    comparison_figure_path  
)
```



Saved scenario comparison figure: C:\Users\User\Documents\GitHub\m66-passenger-queue-simulation\MODESIM_Project\output\figures\scenario_comparison_summary.png

21. Stop-Level Results

Period-level results show the overall performance of each operating condition, but they do not identify where queue pressure is concentrated within the five-stop study segment.

This section summarizes the simulation results separately for Stop Sequences 6 through 10 under Baseline, High-Demand, Reduced-Service, and Combined conditions.

For each scenario, study period, and stop, the table reports the mean across retained replications for completed passengers, waiting time, peak queue, period-end queue, and final backlog. The reported P95 value is the mean of the replication-level stop P95 waiting times rather than a pooled passenger-level percentile.

The highest mean peak-queue stop is also identified for each scenario and study period to provide a compact indication of where queue pressure is greatest.

```
In [43]: # Combine the stop-level results from all four simulation conditions.
all_stop_level_results = pd.concat(
    [
```

```

        baseline_stop_df,
        high_demand_stop_df,
        reduced_service_stop_df,
        combined_stop_df
    ],
    ignore_index=True
)

# Build a lookup so the stop-level outputs include both the stop
# sequence and the corresponding stop name.
stop_name_lookup = (
    demand_table[
        ["Stop Sequence", "Stop Name"]
    ]
    .drop_duplicates()
    .set_index("Stop Sequence")["Stop Name"]
    .to_dict()
)

# Summarize the stop-level metrics across retained replications.
stop_level_summary_df = (
    all_stop_level_results
    .groupby(
        [
            "Scenario",
            "Period",
            "Stop Sequence"
        ]
    )
    .agg(
        Replications=(
            "Replication",
            "nunique"
        ),
        **{
            "Mean Completed Passengers": (
                "Completed Passengers",
                "mean"
            ),
            "Mean Wait (minutes)": (
                "Mean Wait",

```

```

        "mean"
    ),
    "Mean Replication P95 Wait (minutes)": (
        "P95 Wait",
        "mean"
    ),
    "Mean Peak Queue (passengers)": (
        "Peak Queue",
        "mean"
    ),
    "Maximum Peak Queue Observed": (
        "Peak Queue",
        "max"
    ),
    "Mean Period-End Queue (passengers)": (
        "Period-End Queue",
        "mean"
    ),
    "Mean Final Backlog (passengers)": (
        "Final Backlog",
        "mean"
    )
    }
)
.reset_index()

# Add the corresponding stop name to each summarized result.
stop_level_summary_df["Stop Name"] = (
    stop_level_summary_df["Stop Sequence"]
    .map(stop_name_lookup)
)

# Apply the same scenario and period ordering used in the
# period-level comparison.
stop_level_summary_df["Scenario"] = pd.Categorical(
    stop_level_summary_df["Scenario"],
    categories=scenario_order,
    ordered=True
)

```

```

stop_level_summary_df["Period"] = pd.Categorical(
    stop_level_summary_df["Period"],
    categories=period_order,
    ordered=True
)

stop_level_summary_df = (
    stop_level_summary_df
    .sort_values(
        [
            "Scenario",
            "Period",
            "Stop Sequence"
        ]
    )
    .reset_index(drop=True)
)

# Arrange the reviewer-facing columns in a consistent order.
stop_level_summary_df = stop_level_summary_df[
    [
        "Scenario",
        "Period",
        "Stop Sequence",
        "Stop Name",
        "Replications",
        "Mean Completed Passengers",
        "Mean Wait (minutes)",
        "Mean Replication P95 Wait (minutes)",
        "Mean Peak Queue (passengers)",
        "Maximum Peak Queue Observed",
        "Mean Period-End Queue (passengers)",
        "Mean Final Backlog (passengers)"
    ]
]

stop_level_summary_df.round(3)

```

Out[43]:

	Scenario	Period	Stop Sequence	Stop Name	Replications	Mean Completed Passengers	Mean Wait (minutes)	Mean Replication P95 Wait (minutes)	Mean Peak Queue (passengers)	Maximum Peak Queue Observed	Mean Period-End Queue (passengers)
0	Baseline	Morning	6	W 65 ST/CENTRAL PARK WEST	30	113.833	3.449	8.321	7.300	10	2.167
1	Baseline	Morning	7	E 65 ST/5 AV	30	23.000	3.583	7.893	2.333	3	0.200
2	Baseline	Morning	8	MADISON AV/E 65 ST	30	13.000	3.693	7.603	1.800	3	0.200
3	Baseline	Morning	9	MADISON AV/E 67 ST	30	29.000	3.685	8.625	2.833	4	0.200
4	Baseline	Morning	10	E 68 ST/LEXINGTON AV	30	319.000	3.733	9.415	17.267	22	0.833
5	Baseline	Evening	6	W 65 ST/CENTRAL PARK WEST	30	123.933	3.327	6.968	7.467	10	2.067
6	Baseline	Evening	7	E 65 ST/5 AV	30	20.667	3.380	6.878	2.600	4	0.333
7	Baseline	Evening	8	MADISON AV/E 65 ST	30	18.667	3.656	8.073	2.200	3	0.333
8	Baseline	Evening	9	MADISON AV/E 67 ST	30	23.767	3.504	7.566	2.600	5	0.233
9	Baseline	Evening	10	E 68 ST/LEXINGTON AV	30	85.733	4.222	11.613	9.667	13	0.267
10	High-Demand	Morning	6	W 65 ST/CENTRAL PARK WEST	30	162.133	3.436	8.367	9.267	12	2.867
11	High-Demand	Morning	7	E 65 ST/5 AV	30	30.000	3.431	7.789	2.900	4	0.333

	Scenario	Period	Stop Sequence	Stop Name	Replications	Mean Completed Passengers	Mean Wait (minutes)	Mean Replication P95 Wait (minutes)	Mean Peak Queue (passengers)	Maximum Peak Queue Observed	Mean Period-End Queue (passengers)
12	High-Demand	Morning	8	MADISON AV/E 65 ST	30	17.000	3.651	7.664	2.133	3	0.300
13	High-Demand	Morning	9	MADISON AV/E 67 ST	30	40.000	3.729	8.843	3.467	5	0.367
14	High-Demand	Morning	10	E 68 ST/LEXINGTON AV	30	446.000	3.755	9.449	23.367	30	1.167
15	High-Demand	Evening	6	W 65 ST/CENTRAL PARK WEST	30	174.200	3.313	6.937	9.533	12	2.800
16	High-Demand	Evening	7	E 65 ST/5 AV	30	27.667	3.379	6.976	2.967	4	0.333
17	High-Demand	Evening	8	MADISON AV/E 65 ST	30	26.567	3.569	7.890	2.800	4	0.433
18	High-Demand	Evening	9	MADISON AV/E 67 ST	30	32.700	3.586	8.254	3.333	6	0.300
19	High-Demand	Evening	10	E 68 ST/LEXINGTON AV	30	119.600	4.210	11.591	13.900	18	0.400
20	Reduced-Service	Morning	6	W 65 ST/CENTRAL PARK WEST	30	113.833	4.608	13.306	9.700	15	2.167
21	Reduced-Service	Morning	7	E 65 ST/5 AV	30	23.000	4.777	12.550	2.600	5	0.200
22	Reduced-Service	Morning	8	MADISON AV/E 65 ST	30	13.000	5.298	12.563	2.067	3	0.200

	Scenario	Period	Stop Sequence	Stop Name	Replications	Mean Completed Passengers	Mean Wait (minutes)	Mean Replication P95 Wait (minutes)	Mean Peak Queue (passengers)	Maximum Peak Queue Observed	Mean Period-End Queue (passengers)
23	Reduced-Service	Morning	9	MADISON AV/E 67 ST	30	29.000	4.985	13.395	3.200	5	0.200
24	Reduced-Service	Morning	10	E 68 ST/LEXINGTON AV	30	319.000	4.911	14.704	24.500	33	0.833
25	Reduced-Service	Evening	6	W 65 ST/CENTRAL PARK WEST	30	123.933	4.014	9.846	8.733	11	2.067
26	Reduced-Service	Evening	7	E 65 ST/5 AV	30	20.667	4.226	9.684	2.733	4	0.333
27	Reduced-Service	Evening	8	MADISON AV/E 65 ST	30	18.667	4.222	9.914	2.267	4	0.333
28	Reduced-Service	Evening	9	MADISON AV/E 67 ST	30	23.767	4.341	11.073	2.800	5	0.233
29	Reduced-Service	Evening	10	E 68 ST/LEXINGTON AV	30	85.733	4.950	14.094	9.833	13	0.267
30	Combined	Morning	6	W 65 ST/CENTRAL PARK WEST	30	162.133	4.585	13.364	13.267	19	2.867
31	Combined	Morning	7	E 65 ST/5 AV	30	30.000	4.558	12.449	3.267	6	0.333
32	Combined	Morning	8	MADISON AV/E 65 ST	30	17.000	5.116	12.206	2.433	4	0.300
33	Combined	Morning	9	MADISON AV/E 67 ST	30	40.000	5.025	13.596	3.933	5	0.367
34	Combined	Morning	10	E 68 ST/LEXINGTON	30	446.000	4.889	14.608	32.900	43	1.167

	Scenario	Period	Stop Sequence	Stop Name	Replications	Mean Completed Passengers	Mean Wait (minutes)	Mean Replication P95 Wait (minutes)	Mean Peak Queue (passengers)	Maximum Peak Queue Observed	Mean Period-End Queue (passengers)
				AV							
35	Combined	Evening	6	W 65 ST/CENTRAL PARK WEST	30	174.200	3.990	9.777	11.633	16	2.800
36	Combined	Evening	7	E 65 ST/5 AV	30	27.667	4.129	9.863	3.300	5	0.333
37	Combined	Evening	8	MADISON AV/E 65 ST	30	26.567	4.134	9.898	2.867	5	0.433
38	Combined	Evening	9	MADISON AV/E 67 ST	30	32.700	4.400	11.501	3.567	6	0.300
39	Combined	Evening	10	E 68 ST/LEXINGTON AV	30	119.600	4.898	14.066	14.000	18	0.400

```
In [44]: # Validate that every scenario, period, and modeled stop is represented
# by the complete set of replications retained for that scenario.
expected_stop_summary_rows = (
    len(scenario_order)
    * len(period_order)
    * len(selected_stops)
)

stop_level_structure_ok = (
    len(stop_level_summary_df)
    == expected_stop_summary_rows
)

expected_replications_by_scenario = {
    "Baseline": baseline_replications_used,
    "High-Demand": high_demand_replications_used,
    "Reduced-Service": reduced_service_replications_used,
```

```

        "Combined": combined_replications_used
    }

    expected_stop_replications = (
        stop_level_summary_df[
            "Scenario"
        ]
        .astype(str)
        .map(
            expected_replications_by_scenario
        )
    )

    stop_level_replications_ok = (
        stop_level_summary_df[
            "Replications"
        ].to_numpy()
        == expected_stop_replications.to_numpy()
    ).all()

    stop_names_complete = (
        stop_level_summary_df[
            "Stop Name"
        ]
        .notna()
        .all()
    )

    stop_sequences_complete = (
        set(
            stop_level_summary_df[
                "Stop Sequence"
            ].unique()
        )
        == set(selected_stops)
    )

    stop_level_validation_df = pd.DataFrame([
        {
            "Check": (
                "All scenario-period-stop combinations are present"
            )
        }
    ])

```

```

    ),
    "Passed": stop_level_structure_ok
},
{
    "Check": (
        "Stop-level replication counts match each "
        "scenario's retained replications"
    ),
    "Passed": bool(
        stop_level_replications_ok
    )
},
{
    "Check": (
        "Every stop sequence has a corresponding stop name"
    ),
    "Passed": stop_names_complete
},
{
    "Check": (
        "Stop Sequences 6 through 10 are all represented"
    ),
    "Passed": stop_sequences_complete
}
])

assert stop_level_validation_df[
    "Passed"
].all(), (
    "Stop-level result validation failed."
)

print(
    "Stop-level result structure valid:",
    stop_level_validation_df[
        "Passed"
    ].all()
)

stop_level_validation_df

```

Stop-level result structure valid: True

Out[44]:

		Check	Passed
0	All scenario-period-stop combinations are present		True
1	Stop-level replication counts match each scena...		True
2	Every stop sequence has a corresponding stop name		True
3	Stop Sequences 6 through 10 are all represented		True

In [45]: *# Identify the stop with the largest mean peak queue within each
scenario and study period.*

```
highest_peak_queue_rows = (  
    stop_level_summary_df  
    .groupby(  
        ["Scenario", "Period"],  
        observed=True  
    )  
    ["Mean Peak Queue (passengers)"]  
    .idxmax()  
)  
  
highest_pressure_stop_df = (  
    stop_level_summary_df.loc[  
        highest_peak_queue_rows,  
        [  
            "Scenario",  
            "Period",  
            "Stop Sequence",  
            "Stop Name",  
            "Mean Peak Queue (passengers)",  
            "Maximum Peak Queue Observed",  
            "Mean Wait (minutes)",  
            "Mean Final Backlog (passengers)"  
        ]  
    )  
    .sort_values(  
        ["Scenario", "Period"]  
    )  
)
```

```

    )
    .reset_index(drop=True)
)

highest_pressure_stop_df.round(3)

```

Out[45]:

	Scenario	Period	Stop Sequence	Stop Name	Mean Peak Queue (passengers)	Maximum Peak Queue Observed	Mean Wait (minutes)	Mean Final Backlog (passengers)
0	Baseline	Morning	10	E 68 ST/LEXINGTON AV	17.267	22	3.733	0.000
1	Baseline	Evening	10	E 68 ST/LEXINGTON AV	9.667	13	4.222	0.267
2	High-Demand	Morning	10	E 68 ST/LEXINGTON AV	23.367	30	3.755	0.000
3	High-Demand	Evening	10	E 68 ST/LEXINGTON AV	13.900	18	4.210	0.400
4	Reduced-Service	Morning	10	E 68 ST/LEXINGTON AV	24.500	33	4.911	0.000
5	Reduced-Service	Evening	10	E 68 ST/LEXINGTON AV	9.833	13	4.950	0.267
6	Combined	Morning	10	E 68 ST/LEXINGTON AV	32.900	43	4.889	0.000
7	Combined	Evening	10	E 68 ST/LEXINGTON AV	14.000	18	4.898	0.400

In [46]:

```

# Plot mean peak queue by modeled stop for each scenario and study period.
for period in period_order:
    period_stop_summary = (
        stop_level_summary_df[
            stop_level_summary_df["Period"]
            == period
        ]
        .copy()
    )

```

```

)

period_stop_summary["Stop Label"] = (
    period_stop_summary[
        "Stop Sequence"
    ].astype(str)
    + " - "
    + period_stop_summary[
        "Stop Name"
    ]
)

peak_queue_pivot = (
    period_stop_summary
    .pivot(
        index="Stop Label",
        columns="Scenario",
        values="Mean Peak Queue (passengers)"
    )
    .reindex(
        columns=scenario_order
    )
)

ax = peak_queue_pivot.plot(
    kind="bar",
    figsize=(11, 5)
)

ax.set_title(
    f"{period} Mean Peak Queue by Stop"
)
ax.set_xlabel("Modeled stop")
ax.set_ylabel(
    "Mean peak queue (passengers)"
)
ax.tick_params(
    axis="x",
    rotation=45
)

```

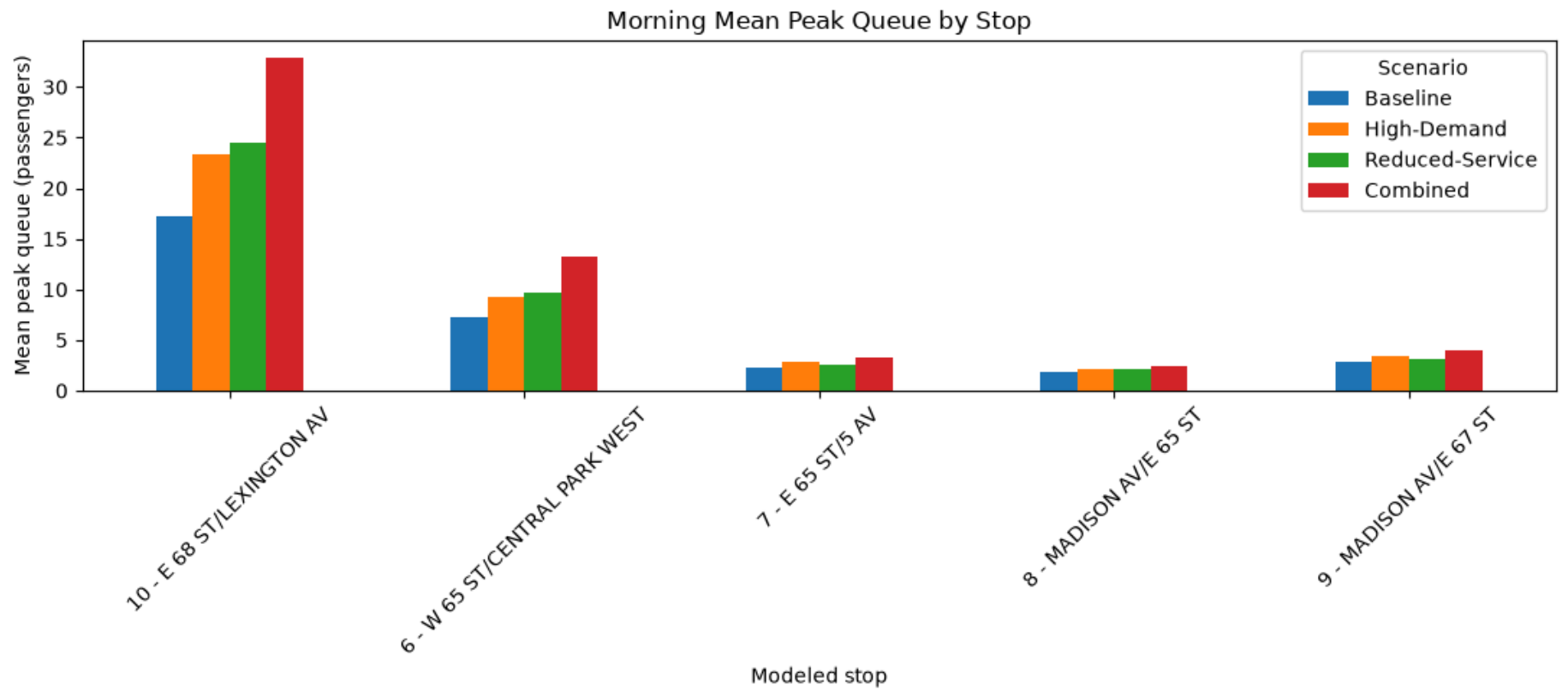
```
plt.tight_layout()

figure_path = (
    figures_dir
    / (
        "stop_level_peak_queue_"
        f"{period.lower()}.png"
    )
)

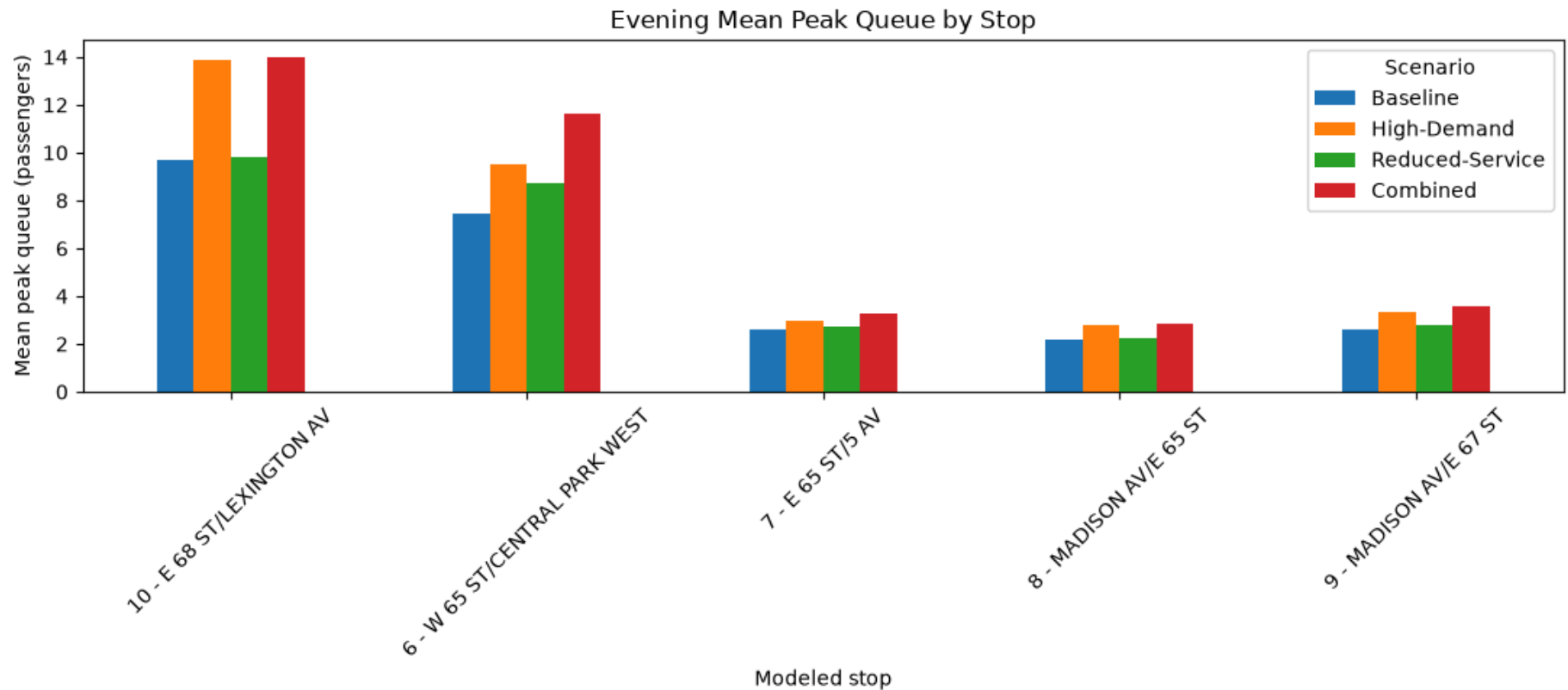
plt.savefig(
    figure_path,
    dpi=300,
    bbox_inches="tight"
)

plt.show()

print(
    f"Saved {period} stop-level figure:",
    figure_path
)
```



Saved Morning stop-level figure: C:\Users\User\Documents\GitHub\m66-passenger-queue-simulation\MODESIM_Project\output\figures\stop_level_peak_queue_morning.png



Saved Evening stop-level figure: C:\Users\User\Documents\GitHub\m66-passenger-queue-simulation\MODESIM_Project\output\figures\stop_level_peak_queue_evening.png

22. Export Results

The completed scenario experiment exports the processed scenario inputs, replication-level results, summary tables, precision checks, and validation evidence used for review and downstream optimization.

Processed scenario-specific datasets are written to `data/processed`. Reviewer-facing summaries and validation tables are written to `output/tables`. Figures generated in the preceding sections are written to `output/figures`.

The exported results include all four operating conditions: Baseline, High-Demand, Reduced-Service, and Combined.

```

In [47]: # Combine the precision results from all four executed scenarios.
all_precision_results_df = pd.concat(
    [
        baseline_precision_df,
        high_demand_precision_df,
        reduced_service_precision_df,
        combined_precision_df
    ],
    ignore_index=True
)

# Keep a compact descriptive difference table for the three stress
# scenarios relative to the matching Baseline period.
scenario_change_export_df = scenario_change_df[
    [
        "Scenario",
        "Period",
        "Difference in Mean Wait",
        "Difference in Mean P95 Wait",
        "Difference in Mean Peak Queue",
        "Difference in Mean Period-End Queue",
        "Difference in Mean Final Backlog",
        "Difference in Mean Bus Occupancy"
    ]
].copy()

# Define scenario-specific processed datasets and replication-level
# outputs that support reproducibility and downstream analysis.
processed_exports = {
    "M66_HighDemand_Demand_Table.csv":
        high_demand_demand,
    "M66_HighDemand_Additional_Demand.csv":
        additional_demand,
    "M66_HighDemand_Master_Passenger_Streams.csv":
        high_demand_master_streams,
    "M66_ReducedService_Service_Plan.csv":
        reduced_service_plan,
    "M66_ReducedService_Removed_Trips.csv":
        removed_trips_df,
    "M66_HighDemand_Results.csv":

```

```

        high_demand_results_df,
    "M66_ReducedService_Results.csv":
        reduced_service_results_df,
    "M66_Combined_Results.csv":
        combined_results_df,
    "M66_HighDemand_StopLevel_Results.csv":
        high_demand_stop_df,
    "M66_ReducedService_StopLevel_Results.csv":
        reduced_service_stop_df,
    "M66_Combined_StopLevel_Results.csv":
        combined_stop_df,
    "M66_HighDemand_Precision_Check.csv":
        high_demand_precision_df,
    "M66_ReducedService_Precision_Check.csv":
        reduced_service_precision_df,
    "M66_Combined_Precision_Check.csv":
        combined_precision_df
}

```

Define reviewer-facing summary and validation tables.

```

table_exports = {
    "scenario_configuration_parameters.csv":
        scenario_config_df,
    "scenario_hourly_trip_counts.csv":
        hourly_trip_counts_with_total,
    "scenario_reducedservice_removal_validation.csv":
        removal_validation_df,
    "scenario_validation_summary.csv":
        validation_summary_df,
    "scenario_matched_seed_validation.csv":
        matched_seed_validation_df,
    "scenario_passenger_demand_validation.csv":
        scenario_passenger_validation_df,
    "scenario_configuration_validation.csv":
        configuration_validation_df,
    "scenario_all_replications.csv":
        all_scenario_results,
    "scenario_comparison_summary.csv":
        scenario_summary_df,
    "scenario_descriptive_differences_from_baseline.csv":
        scenario_change_export_df,
}

```

```

    "scenario_precision_summary.csv":
        all_precision_results_df,
    "scenario_stop_level_all_replications.csv":
        all_stop_level_results,
    "scenario_stop_level_summary.csv":
        stop_level_summary_df,
    "scenario_highest_pressure_stops.csv":
        highest_pressure_stop_df,
    "scenario_stop_level_validation.csv":
        stop_level_validation_df
}

# Write processed scenario datasets.
for file_name, export_df in processed_exports.items():
    export_df.to_csv(
        processed_dir / file_name,
        index=False
    )

# Write summary and validation tables.
for file_name, export_df in table_exports.items():
    export_df.to_csv(
        tables_dir / file_name,
        index=False
    )

print(
    "Processed files exported:",
    len(processed_exports)
)
print(
    "Table files exported:",
    len(table_exports)
)

```

Processed files exported: 14

Table files exported: 15

```

In [48]: # Verify that every expected processed and reviewer-facing table was
# successfully written and contains data.
export_validation_records = []

```

```

for file_name, export_df in processed_exports.items():
    file_path = processed_dir / file_name

    export_validation_records.append({
        "Output Type": "Processed",
        "File": file_name,
        "Exists": file_path.exists(),
        "Rows": len(export_df),
        "Non-Empty": (
            file_path.exists()
            and len(export_df) > 0
        )
    })

for file_name, export_df in table_exports.items():
    file_path = tables_dir / file_name

    export_validation_records.append({
        "Output Type": "Table",
        "File": file_name,
        "Exists": file_path.exists(),
        "Rows": len(export_df),
        "Non-Empty": (
            file_path.exists()
            and len(export_df) > 0
        )
    })

export_validation_df = pd.DataFrame(
    export_validation_records
)

all_exports_valid = bool(
    export_validation_df[
        ["Exists", "Non-Empty"]
    ]
    .all(axis=1)
    .all()
)

```

```
assert all_exports_valid, (  
    "At least one expected scenario output was not "  
    "successfully exported."  
)  
  
print(  
    "All expected CSV outputs exported successfully:",  
    all_exports_valid  
)  
  
export_validation_df
```

All expected CSV outputs exported successfully: True

Out[48]:

	Output Type	File	Exists	Rows	Non-Empty
0	Processed	M66_HighDemand_Demand_Table.csv	True	50	True
1	Processed	M66_HighDemand_Additional_Demand.csv	True	50	True
2	Processed	M66_HighDemand_Master_Passenger_Streams.csv	True	32490	True
3	Processed	M66_ReducedService_Service_Plan.csv	True	10	True
4	Processed	M66_ReducedService_Removed_Trips.csv	True	10	True
5	Processed	M66_HighDemand_Results.csv	True	60	True
6	Processed	M66_ReducedService_Results.csv	True	60	True
7	Processed	M66_Combined_Results.csv	True	60	True
8	Processed	M66_HighDemand_StopLevel_Results.csv	True	300	True
9	Processed	M66_ReducedService_StopLevel_Results.csv	True	300	True
10	Processed	M66_Combined_StopLevel_Results.csv	True	300	True
11	Processed	M66_HighDemand_Precision_Check.csv	True	4	True
12	Processed	M66_ReducedService_Precision_Check.csv	True	4	True
13	Processed	M66_Combined_Precision_Check.csv	True	4	True
14	Table	scenario_configuration_parameters.csv	True	4	True
15	Table	scenario_hourly_trip_counts.csv	True	11	True
16	Table	scenario_reducedservice_removal_validation.csv	True	5	True
17	Table	scenario_validation_summary.csv	True	4	True
18	Table	scenario_matched_seed_validation.csv	True	4	True
19	Table	scenario_passenger_demand_validation.csv	True	4	True
20	Table	scenario_configuration_validation.csv	True	5	True
21	Table	scenario_all_replications.csv	True	240	True

	Output Type	File	Exists	Rows	Non-Empty
22	Table	scenario_comparison_summary.csv	True	8	True
23	Table	scenario_descriptive_differences_from_baseline...	True	6	True
24	Table	scenario_precision_summary.csv	True	16	True
25	Table	scenario_stop_level_all_replications.csv	True	1200	True
26	Table	scenario_stop_level_summary.csv	True	40	True
27	Table	scenario_highest_pressure_stops.csv	True	8	True
28	Table	scenario_stop_level_validation.csv	True	4	True

```
In [49]: # Build a compact manifest showing where the scenario notebook writes
# its processed data, summary tables, and figures.
output_manifest_records = []
```

```
for file_name in processed_exports:
    output_manifest_records.append({
        "Category": "Processed Data",
        "File": file_name,
        "Directory": "data/processed"
    })

for file_name in table_exports:
    output_manifest_records.append({
        "Category": "Table",
        "File": file_name,
        "Directory": "output/tables"
    })

for figure_name in [
    "scenario_comparison_summary.png",
    "stop_level_peak_queue_morning.png",
    "stop_level_peak_queue_evening.png"
]:
    figure_path = figures_dir / figure_name
```

```

        output_manifest_records.append({
            "Category": "Figure",
            "File": figure_name,
            "Directory": "output/figures"
        })

output_manifest_df = pd.DataFrame(
    output_manifest_records
)

output_manifest_path = (
    tables_dir
    / "scenario_output_manifest.csv"
)

output_manifest_df.to_csv(
    output_manifest_path,
    index=False
)

print(
    "Output manifest saved:",
    output_manifest_path
)

output_manifest_df

```

Output manifest saved: C:\Users\User\Documents\GitHub\m66-passenger-queue-simulation\MODESIM_Project\output\tables\scenario_output_manifest.csv

Out[49]:

	Category	File	Directory
0	Processed Data	M66_HighDemand_Demand_Table.csv	data/processed
1	Processed Data	M66_HighDemand_Additional_Demand.csv	data/processed
2	Processed Data	M66_HighDemand_Master_Passenger_Streams.csv	data/processed
3	Processed Data	M66_ReducedService_Service_Plan.csv	data/processed
4	Processed Data	M66_ReducedService_Removed_Trips.csv	data/processed
5	Processed Data	M66_HighDemand_Results.csv	data/processed
6	Processed Data	M66_ReducedService_Results.csv	data/processed
7	Processed Data	M66_Combined_Results.csv	data/processed
8	Processed Data	M66_HighDemand_StopLevel_Results.csv	data/processed
9	Processed Data	M66_ReducedService_StopLevel_Results.csv	data/processed
10	Processed Data	M66_Combined_StopLevel_Results.csv	data/processed
11	Processed Data	M66_HighDemand_Precision_Check.csv	data/processed
12	Processed Data	M66_ReducedService_Precision_Check.csv	data/processed
13	Processed Data	M66_Combined_Precision_Check.csv	data/processed
14	Table	scenario_configuration_parameters.csv	output/tables
15	Table	scenario_hourly_trip_counts.csv	output/tables
16	Table	scenario_reducedservice_removal_validation.csv	output/tables
17	Table	scenario_validation_summary.csv	output/tables
18	Table	scenario_matched_seed_validation.csv	output/tables
19	Table	scenario_passenger_demand_validation.csv	output/tables
20	Table	scenario_configuration_validation.csv	output/tables
21	Table	scenario_all_replications.csv	output/tables

	Category	File	Directory
22	Table	scenario_comparison_summary.csv	output/tables
23	Table	scenario_descriptive_differences_from_baseline...	output/tables
24	Table	scenario_precision_summary.csv	output/tables
25	Table	scenario_stop_level_all_replications.csv	output/tables
26	Table	scenario_stop_level_summary.csv	output/tables
27	Table	scenario_highest_pressure_stops.csv	output/tables
28	Table	scenario_stop_level_validation.csv	output/tables
29	Figure	scenario_comparison_summary.png	output/figures
30	Figure	stop_level_peak_queue_morning.png	output/figures
31	Figure	stop_level_peak_queue_evening.png	output/figures

Technical Reference

Detailed data, simulation, optimization, validation, and methodological references are documented in

`../docs/TECHNICAL_DOCUMENTATION.md`.